

PROJECT SUBMISSION REPORT

Libris

A Role-Based Library Management System

A layered Spring Boot 3.5 / MySQL application for catalogue, membership, and borrow-return management — documenting architecture, security, testing, deployment, and an end-to-end software-maintenance and bug-fixing case log.

Submitted by	Pratham Kashyap
Application No.	IN26009772
Registration No.	23BAII0455
Program	Advanced Software Engineering & Development Internship (SE3)
Batch	12B
Repository	Library-Management-System
Date	July 2026

Table of Contents

Table of Contents	2
1. Executive Summary.....	7
2. Introduction & Objectives.....	7
2.1 Project Overview	7
2.2 Objectives.....	7
2.3 Scope.....	8
3. Development Process & Timeline	9
3.1 Delivery Timeline.....	9
3.2 Post-Release Growth.....	11
4. Requirements Traceability	12
4.1 Functional Requirements.....	12
4.2 Non-Functional Requirements.....	13
5. System Architecture	14
6. Design Rationale & Alternatives Considered	15
4.1 Technology Alternatives Rejected.....	15
4.2 Database Design Alternatives.....	15
7. Technology Stack — Tools Used and How	16
9. Database Design.....	20
10. API Design & Reference.....	25
11. Security Architecture	30
11.1 Login Flow	30
11.2 CSRF Protection	30
11.3 Authorization Model.....	30
11.4 Security Handlers & Password Storage	32
11.5 Security Considerations for Future Hardening	32
12. Frontend Architecture & Screen Reference	34
12.1 Screen Reference	34
12.3 Design System	36
12.4 Security in the Frontend	36
12.5 Frontend Considerations for Future Enhancement	37
13. Software Engineering Practices Applied	38
13.1 Mapping to Software Maintenance Types.....	38
14. Testing & Quality Assurance	39
14.1 Test Inventory.....	39
14.2 Current Test Coverage Scope	39
14.3 Manual / Black-Box Testing.....	40
14.4 Planned Test Coverage Expansion.....	41
15. Quality Assurance & Issue Resolution Log	42

15.1 Defect Log.....	42
15.2 Case Studies	42
16. Problems Faced & Resolutions	46
17. Deployment & Operations	47
18. Results & Outcomes	49
18.1 Delivered Scope Summary.....	49
18.2 Interface Reference	51
19. Recommendations & Future Enhancements	54
20. Conclusion	54
Appendix A: Complete API Endpoint Index	56
Appendix B: Glossary	59

CERTIFICATE

This is to certify that the project report entitled "**Libris: A Role-Based Library Management System**" is a bonafide work carried out by **Pratham Kashyap** in partial fulfilment of the requirements for the **Advanced Software Engineering & Development Internship (SE3)**.

The work presented in this report has been completed under proper guidance and supervision. It embodies the original work of the candidate and has not been submitted, either in part or in full, to any other university, institute, or organization for the award of any degree, diploma, or certification.

The project demonstrates the successful design and implementation of a secure web-based Library Management System using modern software engineering principles, including layered architecture, RESTful APIs, role-based authentication and authorization, database management, testing, deployment, and documentation.

Project Title:

Libris: A Role-Based Library Management System

Guide:

Name: **Mr. Rupesh Phalne**

Institution: **MPOnline Limited**

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide and all the faculty members of the **Advanced Software Engineering & Development Internship (SE3)** for their continuous guidance, encouragement, and valuable suggestions throughout the development of this project.

I would also like to thank the internship mentors for providing an excellent learning environment that enabled me to enhance my practical knowledge of modern software development using Java, Spring Boot, MySQL, Spring Security, RESTful APIs, Docker, and GitHub Actions.

My heartfelt appreciation goes to my family and friends for their constant motivation, encouragement, and support during every stage of this project.

Special thanks to the open-source community whose documentation and development tools, including Spring Boot, Hibernate, Maven, MySQL, Docker, GitHub, and related technologies, played a significant role in the successful completion of this project.

Finally, I thank everyone who directly or indirectly contributed to making this project successful and helping me gain valuable practical experience in full-stack software development.

ABSTRACT

Library management in many educational institutions and small organizations still relies on manual record keeping or isolated desktop applications, resulting in inefficient catalogue management, delayed book tracking, inconsistent records, and limited access control. To overcome these challenges, this project presents **Libris**, a secure web-based **Role-Based Library Management System** developed using **Spring Boot 3.5, Java 21, MySQL, Spring Security, Hibernate (JPA), and RESTful APIs**.

The system follows a layered MVC architecture consisting of Controller, Service, Repository, and Persistence layers to ensure modularity, maintainability, and scalability. It provides role-specific access for **Administrators, Librarians, and Students**, enabling secure authentication, catalogue management, membership management, borrow-return operations, dashboards, analytics, audit logging, and CSV report generation. The application also integrates Google OAuth2 authentication, session-based security, CSRF protection, and BCrypt password encryption to strengthen application security.

The project emphasizes software engineering best practices through DTO-based API design, transactional database operations, exception handling, automated and manual testing, Docker-based deployment, and continuous integration using GitHub Actions. The developed system successfully manages books, magazines, and newspapers while maintaining complete borrowing history and preventing unauthorized access through role-based authorization.

The implemented solution improves operational efficiency, minimizes manual effort, ensures data consistency, enhances system security, and provides a scalable foundation for future enhancements such as reservation management, notifications, online fine payment, pagination, and advanced reporting. Overall, Libris demonstrates the practical application of modern enterprise software engineering principles for developing a reliable, secure, and maintainable Library Management System.

1. Executive Summary

Libris is a layered Spring Boot 3.5 (Java 21) application built to run a small library's day-to-day operations: a multi-type catalogue (books, magazines, newspapers), three tiers of user accounts, and a full borrow/return lifecycle, wrapped behind a session-authenticated REST API and a hand-built multi-page front end. Over the course of development the system grew well past its original book-only scope — it now also keeps a persistent audit trail of every mutating action, exposes role-specific analytics and dashboards, generates CSV reports, and supports Google OAuth2 login alongside the original username/password flow, all shipped with Docker Compose packaging and a GitHub Actions CI pipeline.

This report documents the completed system exactly as delivered, built directly from the project's own current architecture, security, database, frontend, testing, and deployment documentation. It covers the full engineering picture — the development timeline and requirements traceability, architecture and design rationale, technology choices and how each was used, module-by-module implementation, database and API design, security architecture, frontend structure, the automated and manual testing strategy, and a complete, closed defect log — before closing with delivered results and a forward-looking roadmap for future enhancement.

At a glance

8 JPA entities · **14** REST controllers · **48** documented API endpoints · **11** services · **26** DTOs
18 front-end pages · **3** roles (Admin, Librarian, Student) · **6** automated test methods · **14**
documented manual black-box cases

2. Introduction & Objectives

2.1 Project Overview

Small and mid-sized libraries frequently rely on spreadsheets or paper registers to track loans, which makes it hard to answer simple questions — which items are out, who has them, and when they are due back. Libris replaces that manual process with a role-secured web application: librarians and administrators manage the catalogue and membership records, students borrow and return items through a self-service view, and every borrow and return is recorded as a durable, queryable transaction rather than a note in a register.

2.2 Objectives

- Provide secure, role-differentiated access for Administrators, Librarians, and Students, with server-side enforcement of what each role may see and do.
- Support full lifecycle management of books, magazines, and newspapers — including search, availability tracking, and safe deletion that will not silently destroy loan history.
- Model the borrow/return cycle as an atomic, transactional operation that keeps item availability and loan records consistent at all times.
- Expose a documented, DTO-bounded REST API so the front end never depends on internal entity shapes or receives sensitive fields such as password hashes.
- Give staff an at-a-glance operational picture through dashboards and analytics, and preserve a full audit trail of catalogue and membership changes.

- Validate the system with an automated test suite and a documented manual test matrix, and capture every non-trivial defect found along the way as a reviewable case study.

2.3 Scope

The delivered system covers authentication (session-based, with an added Google OAuth2 option), book/magazine/newspaper catalogue management, student and librarian account management, the borrow/return workflow, general and role-specific dashboards, analytics, an audit log, CSV reporting, and a self-service profile view. Categories, pagination, fines, notifications, JWT, and reservations were consciously left out of this iteration under the project's frozen scope boundaries.

3. Development Process & Timeline

Libris was built iteratively over a structured, day-by-day delivery plan, with each milestone verified before the next began. This section reconstructs that delivery history from the project's own CHANGELOG, showing the order in which the architecture, persistence layer, APIs, security, and frontend were assembled and hardened.

3.1 Delivery Timeline

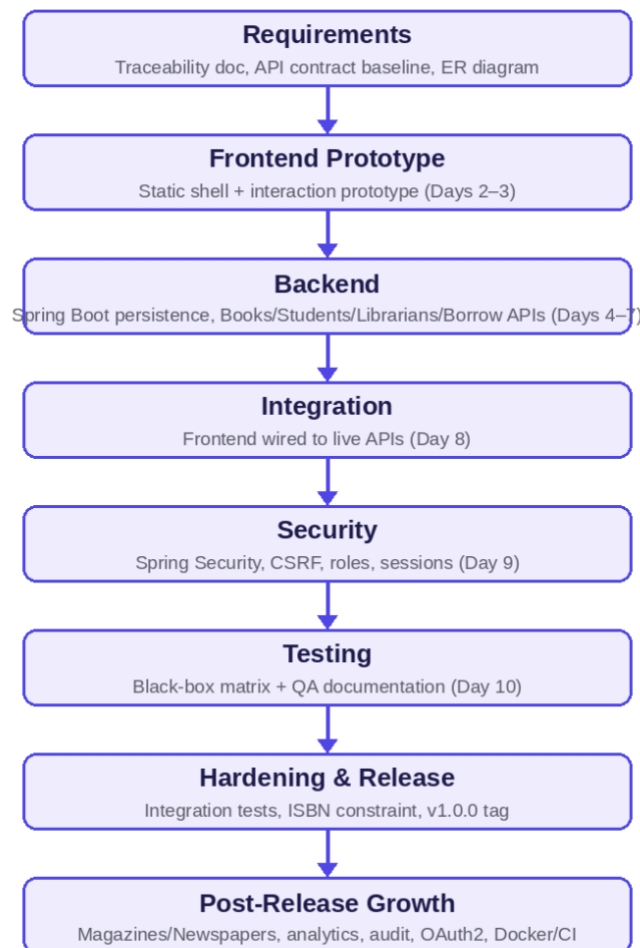


Figure 3.1: High-level delivery flow from requirements through post-release growth. The detailed, milestone-by-milestone breakdown with verification evidence follows in the table below.

Milestone	Delivered	Verified
Day 1 — Architecture baseline	Requirements traceability document, API contract baseline (endpoints, authority, JSON, status, error conventions), architecture entry point, and a Mermaid ER diagram.	API contract uses the canonical /api/borrow-records resource name; schema holds only the approved baseline tables; no undocumented endpoints exist.
Day 2 — Responsive frontend shell	Static Spring Boot frontend shell with Login, Dashboard, Books, Students, Librarians, Borrow Records, and Profile pages; responsive	Temporary demo rendering isolated in js/main.js, explicitly earmarked for replacement once real API integration landed.

Milestone	Delivered	Verified
	navigation; accessible skip link and focus styles; indigo/teal design tokens.	
Day 3 — Frontend interaction prototype	Reusable accessible modal component for book, student, librarian, and borrow forms; client-side required/email validation; toast feedback; book filtering with an empty state; a shared Fetch helper with session credentials and safe 204 handling.	The prototype supports the approved create/borrow flows without adding any deferred field (due dates, categories).
Day 4 — Spring Boot persistence foundation	Java 21 Spring Boot project with Web, Validation, Data JPA, Security, MySQL, and test dependencies; the frozen entities (Account, StudentProfile, LibrarianProfile, Book, BorrowRecord) plus the Role enum and focused repositories; MySQL configuration with environment-overridable credentials.	No controllers, direct SQL, DTO shortcuts, or non-frozen tables were introduced at this stage.
Day 5 — Books API	/api/books CRUD controller, Book request/response DTOs, service-owned search/update/delete behaviour, global structured error responses, and a deletion guard preventing removal of a book with borrow history.	mvn -DskipTests compile passes; endpoint resource and DTO fields match the documented contract; entities are never exposed from controllers.
Day 6 — Students & Librarians APIs	Transactional account/profile creation with BCrypt hashes and unique-username protection; DTO-based list/get/create/update/delete endpoints; separate update DTOs that cannot reset a password.	Only the approved account/profile fields and REST resource names were used; no unapproved password-reset scope was inferred.
Day 7 — Borrow records, dashboard, domain errors	Canonical /api/borrow-records list/borrow/return endpoints; transactional borrow/return services maintaining availability and preserving borrower snapshots; dashboard counts; structured BOOK_UNAVAILABLE / ALREADY_RETURNED errors.	Routes, snapshots, status strings, and dashboard fields match the frozen API contract.
Day 8 — Frontend/backend integration	Centralized Fetch clients for every implemented resource, replacing temporary rendered demo data; UI refresh after create/borrow/return actions; API-backed search; common JSON/error/204 handling.	All approved modules reachable through responsive navigation with no unapproved UI framework introduced.
Day 9 — Spring Security	BCrypt-backed authentication, session login/logout/current-user endpoints, role-based endpoint restrictions, CSRF token forwarding, and a development-only seeded admin account.	Role restrictions verified against the authorization matrix (Section 11.3).

Milestone	Delivered	Verified
Day 10 — QA & delivery documentation	An executable black-box test matrix and complete local run instructions.	Java compilation, JavaScript syntax checks, and whitespace checks all pass.
Hardening pass	H2-backed MockMvc integration tests for authentication, CRUD, borrow/return, validation, logout, and JSON 401/403 failures; a repository test for ISBN uniqueness and auditing timestamps; structured JSON auth/authorization handlers; AuthService; explicit lazy relationship mappings; database-level ISBN uniqueness; Spring Data auditing.	mvn test passes end-to-end against an isolated H2 database in MySQL-compatibility mode.
Release preparation (v1.0.0)	Duplicate-ISBN conflict handling, field-level email validation, restored indigo/teal branding across navigation and the sign-in view, consolidated documentation under docs/, and this release's CHANGELOG.	All release-blocking defects (Section 15) verified closed prior to tagging v1.0.0.

3.2 Post-Release Growth

Following the v1.0.0 baseline, the catalogue and operational surface were extended in a second phase — magazines and newspapers joined the catalogue, role-specific Student and Librarian dashboards were added, an audit trail and analytics module were introduced, CSV reporting shipped, Google OAuth2 became available as a second login path, and the deployment story was completed with a multi-stage Docker build, Docker Compose, and a GitHub Actions CI pipeline. Each addition followed the same controller → service → repository shape and DTO discipline established in the Day 1–10 baseline, which is why the architecture in Section 5 still reads as one coherent system rather than a patchwork.

4. Requirements Traceability

Every functional and non-functional requirement set at the start of the project is tracked here against the milestone that delivered it and the evidence that confirms it — the same traceability discipline the project has maintained since Day 1.

4.1 Functional Requirements

ID	Requirement	Delivered	Acceptance evidence
FR-01	Users can log in, log out, and obtain safe current-session identity information.	Day 9	Security tests; BB-01, BB-02, BB-14.
FR-02	Admins and librarians can view dashboard totals for students, librarians, books, borrowed, and available items.	Day 7	Dashboard API and UI verification.
FR-03	Authorised staff can create, list, view, update, search, and delete eligible catalogue items.	Day 5 (extended for magazines/newspapers post-release)	Books API tests including validation, not-found, and deletion-conflict cases.
FR-04	Admins and librarians can manage student accounts and profiles atomically.	Day 6	Student CRUD API and account/profile transaction test.
FR-05	Only admins can manage librarian accounts and profiles.	Day 6 / Day 9	Librarian CRUD and role-authorisation test; BB-11.
FR-06	Staff can borrow an available item, creating a durable borrow record and marking it unavailable in one transaction.	Day 7	BB-06 and service/integration test.
FR-07	Staff can return an active borrow record, recording the return date and restoring availability in one transaction.	Day 7	BB-08 and service/integration test.
FR-08	The system rejects an unavailable-item borrow and an already-returned record with documented errors.	Day 7	BB-07 and BB-09.
FR-09	Authenticated users can view their own non-sensitive profile information.	Day 9	Profile API/UI test.
FR-10	The frontend works on desktop and mobile and evolves the reviewed indigo/teal prototype.	Days 2–3, 8	Viewport/manual UI tests; BB-13.

4.2 Non-Functional Requirements

ID	Requirement	Design control
NFR-01	Maintain a layered Browser → Controller → Service → Repository → MySQL architecture.	Package boundaries and code review (Section 5).
NFR-02	Do not expose JPA entities or password hashes through the API.	Request/response DTOs only — 26 dedicated classes (Section 10).
NFR-03	Store only BCrypt password hashes and enforce server-side role checks.	Spring Security, delivered Day 9 (Section 11).
NFR-04	Preserve borrow history and prevent destructive cascades from removing audit data.	FK/delete policy and service-level validation (Section 9).
NFR-05	Return documented HTTP statuses and uniform JSON errors.	Global exception handling and the API contract (Section 10.1).
NFR-06	Treat responsive, keyboard-accessible, readable UI behaviour as a release requirement.	Mobile-first CSS and manual checks (Section 12).
NFR-07	Keep documentation, API contract, tests, and CHANGELOG current with implementation.	Definition of Done and a day-by-day changelog (Section 3).

All ten functional requirements and all seven non-functional requirements are delivered and evidenced as of this report. The scope boundaries agreed at project start — excluding categories, pagination, fines, notifications, JWT, and reservations from this release — are unchanged and are revisited as future enhancements in Section 19.

5. System Architecture

Libris follows a classic layered MVC architecture inside a single deployable Spring Boot process: Browser → Fetch → Controller → Service → Repository → JPA/Hibernate → MySQL. Requests flow one direction down the stack and responses flow back up; the security filter chain sits in front of every layer and inspects every request before it reaches a controller. Controllers never contain direct SQL or core business rules — services own transactions and domain rules.

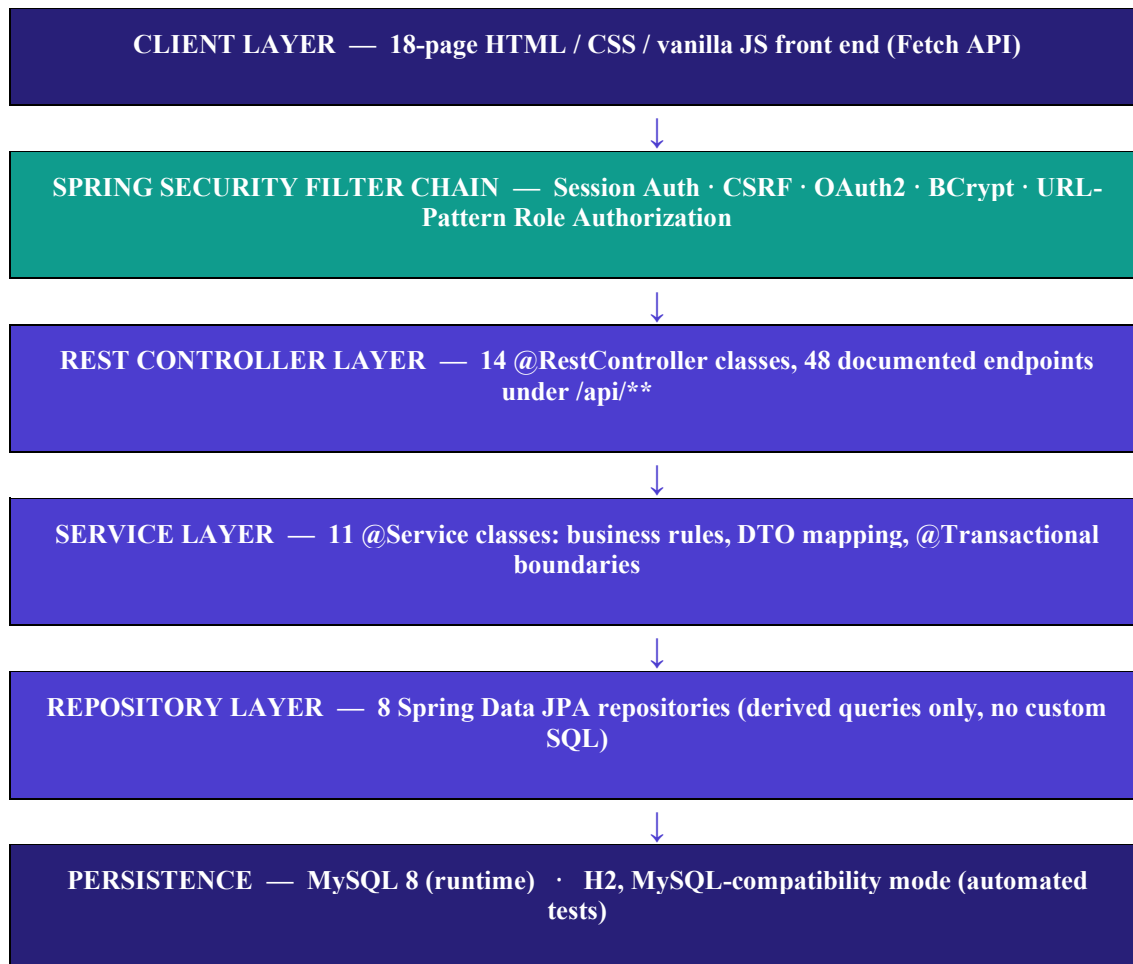


Figure 1: Layered request flow. The security filter chain is cross-cutting — it evaluates every request, authenticated or not, before any controller code runs.

Each layer has one job. Controllers translate HTTP into method calls; services own transactions, validation beyond simple field constraints, and the mapping between JPA entities and the DTOs that actually cross the API boundary; repositories are thin Spring Data interfaces with derived query methods, deliberately avoiding hand-written SQL so that the H2 test database and MySQL stay behaviourally interchangeable. There is no migration tool (Flyway/Liquibase) — schema is generated directly from entity annotations via Hibernate's ddl-auto, a documented gap discussed in Section 14.

6. Design Rationale & Alternatives Considered

Before implementation began, the team reviewed an existing HTML prototype, a set of reference ASP.NET MVC learning materials, meeting screenshots documenting scope evolution, and prior Spring Boot planning notes. What each contributed — and what was deliberately left behind — shaped the frozen architecture.

Source reviewed	Retained	Not carried forward
HTML prototype	Indigo/teal visual language, cards, status badges, modal/toast pattern, client-rendered navigation feel, vanilla JavaScript (no framework)	In-memory data arrays; single-file implementation; the prototype's “due back” field (not in the frozen data model)
Reference ASP.NET MVC materials	Functional scope: book CRUD, borrow forms, availability changes, dashboard counts, student/librarian CRUD	Bootstrap MVC UI, direct SQL in controllers, plaintext/mock login, cascade-style history loss
Meeting screenshots	Confirmed module growth order: Books first, then Students, Librarians, Login, Dashboard, Logout	A referenced Categories module — noted but not adopted as a baseline module
Prior Spring Boot planning notes	Layered services/repositories, JPA, documentation-first rationale	Direct entity exposure; arbitrary frontend framework adoption

4.1 Technology Alternatives Rejected

- **Server-rendered Thymeleaf/JSP views** — rejected because the working prototype was already client-rendered; server templates would have duplicated front-end work for no benefit.
- **React, Angular, Vue, Vaadin** — rejected due to build-pipeline and framework overhead disproportionate to the project's scope; vanilla JavaScript was sufficient.
- **Bootstrap / AdminLTE / CoreUI templates** — rejected to preserve the bespoke prototype's visual identity rather than adopt a generic admin theme.
- **H2 or another embedded database for production** — rejected; MySQL gives realistic foreign-key behaviour and a durable setup. H2 is used for automated tests only.
- **Raw JDBC / direct SQL in controllers** — rejected because it couples HTTP handling, SQL, mapping, and business rules together; Spring Data JPA repositories plus a service layer were selected instead.
- **Direct entity serialization** — rejected because it risks leaking password hashes and internal persistence fields; DTOs are mandatory at the API boundary.
- **JWT for the first release** — deferred; token issuance, storage, expiry, refresh, and revocation add complexity that server-side sessions avoid for a same-origin deployment.

4.2 Database Design Alternatives

A single wide users table was rejected for weakening normalization and making role evolution awkward. A fully separate users/students/librarians table design was not adopted as written, but its underlying insight — keep role-specific data separate — was refined into one shared accounts table with strict one-to-one profile relationships. A borrowed-flag-only design on the catalogue tables was rejected outright: it loses loan history, cannot model

completed loans, and cannot detect a double return. The selected design — separate accounts, student_profiles, librarian_profiles, books/magazines/newspapers, and borrow_records tables — is detailed in Section 8.

7. Technology Stack — Tools Used and How

Every tool below was a deliberate choice tied to a specific requirement rather than a default; the “How it was used” column reflects the actual role each one plays in the running system.

Layer	Tool / Technology	How it was used
Language & Build	Java 21, Maven (via ./mvnw)	Core language; the Maven wrapper pins the build tool version so the project builds identically on every machine without a separate local Maven install.
Web Framework	Spring Boot 3.5 (Web, Validation, Data JPA)	Backs 14 REST controllers and 48 documented endpoints; Bean Validation annotations on request DTOs (@NotBlank, @Email, @Size, @Min/@Max) reject malformed input before it reaches a service.
Security	Spring Security 6.5, BCrypt, OAuth2 Client	Session login/logout, CookieCsrfTokenRepository + a custom SpaCsrfTokenRequestHandler, BCrypt password hashing, and an optional Google OAuth2 sign-in path, all enforced by role at the URL-pattern level.
ORM / Database	Hibernate (ddl-auto=update), MySQL 8	Hibernate maps 8 entities to MySQL tables and evolves the schema automatically in development; MySQL 8/9 is the production/runtime datastore.
Test Database	H2 (MySQL-compatibility mode, ddl-auto=create-drop)	Lets the automated suite exercise real SQL semantics through MockMvc without a live MySQL server; schema is created fresh and dropped every test run.
API Documentation	springdoc-openapi (Swagger UI at /swagger-ui/index.html)	Generates an always-current, interactive API reference directly from controller and DTO annotations.
Front End	HTML5, two-file CSS architecture, vanilla JavaScript (ES modules), Fetch API	18 dedicated pages, each with its own JS module; a shared http.js Fetch helper injects the CSRF header and session credentials on every state-changing call.
Testing	JUnit 5, Spring Boot Test, MockMvc, spring-security-test	Full HTTP-level request/response testing of controllers and the security filter chain without starting a real server or browser.
Containerization	Docker (multi-stage build), Docker Compose	A maven:3.9-eclipse-temurin-21 build stage produces the JAR; the runtime stage uses the slimmer eclipse-temurin:21-jre image. Compose wires the app to a MySQL container with a health check, plus an optional phpMyAdmin profile.

Layer	Tool / Technology	How it was used
CI/CD	GitHub Actions	Runs mvn clean verify on every push/PR to main — compiles the project and runs the full test suite, failing the build on any test failure. No deployment step; deployment is manual.
Version Control	Git	Full source history; CHANGELOG.md is maintained release-by-release alongside commits.

8. Module-Wise Implementation

```

com.libris
├── controller/
│   ├── BookController, StudentController, LibrarianController,
│   ├── BorrowRecordController, AuthController, DashboardController,
│   ├── AnalyticsController, AuditController, ReportController ... (14 tot
├── service/
│   ├── BookService, StudentService, LibrarianService, BorrowService,
│   ├── AuthService, DashboardService ... (11 total, @Transactional)
├── repository/
│   └── 8 Spring Data JPA interfaces – derived queries only
├── model/ (entity)
│   ├── Account, StudentProfile, LibrarianProfile, Book, Magazine,
│   ├── Newspaper, BorrowRecord, AuditLog + AuditableEntity
├── dto/
│   └── 26 request/response DTOs – never expose entities
├── config/
│   └── SecurityConfig, SpaCsrfTokenRequestHandler, AdminSeeder
├── exception/
│   └── GlobalExceptionHandler, ApiErrorResponse, domain exceptions

```

Figure 8.1: Backend package structure. Each layer (controller, service, repository, model, dto, config, exception) has a single, well-defined responsibility, mirrored consistently across every resource.

8.1 Authentication & Session

The frontend calls GET /api/auth/csrf on page load to obtain the XSRF-TOKEN cookie before anything else happens. Session-based login (POST /api/auth/login) issues a server-side HttpSession on successful credential check via a DaoAuthenticationProvider wired to a custom AccountUserDetailsService and BCrypt. Google OAuth2 was added as a second, optional sign-in path (defaultSuccessUrl("/")) alongside the original username/password flow. GET /api/auth/me and GET /api/profile both return the same safe AuthenticatedUserResponse shape — account ID, username, role, and display name, never a password or hash.

8.2 Book, Magazine & Newspaper Catalogue

All three catalogue types follow an identical CRUD shape with search, database-level uniqueness where relevant (ISBN on books), and a deletion guard: BookService, MagazineService, and NewspaperService each check for existing borrow history before allowing a delete and throw a ConflictException (409) if any is found, so history is never silently cascade-deleted. Search on /books matches title OR author, case-insensitive; the frontend also accepts ?query= as an alias for ?search=.

8.3 People Management — Students & Librarians

POST /students and POST /librarians each atomically create an Account (BCrypt-hashed password, correct Role) and a linked profile in one @Transactional method. Only Admins may manage Librarian accounts; Admins and Librarians may both manage Students. Update DTOs (StudentUpdateRequest, LibrarianUpdateRequest) omit the password field entirely — an ordinary profile edit can never silently reset a credential — though username remains mutable and is re-checked for uniqueness on change.

8.4 Borrow / Return Lifecycle

Borrowing and returning are each a single transactional operation. A notable, deliberate API design detail: the BorrowRequest DTO carries borrowerName, borrowerEmail, and borrowerPhone fields that are validated for shape but then overwritten in the service layer with the actual student profile's current data — the request fields exist for API completeness, but the persisted snapshot always reflects the real profile at the moment of borrowing, not

whatever the client happened to send. GET /borrow-records/my lets any authenticated user (including a Student) retrieve their own borrow history without staff-level access to the full list.

8.5 Dashboards

A general GET /dashboard (Admin/Librarian) aggregates totals — students, librarians, books, borrowed, available. A dedicated GET /librarian/dashboard reuses the same DashboardService for a librarian-facing view, and GET /student/dashboard gives each student their own active-borrows-and-history view, scoped to STUDENT, LIBRARIAN, and ADMIN roles.

8.6 Analytics, Audit Trail & Reporting

Five analytics endpoints (dashboard, trends, top-books, top-readers, overdue) aggregate borrowing activity for Admin/Librarian use. An append-only AuditLog entity records actor, action (CREATE/UPDATE/DELETE/LOGIN/LOGOUT/BORROW/RETURN), entity type, and a human-readable description for every mutating action, queryable and filterable by an Admin-only GET /audit. Four report endpoints stream CSV exports (inventory, borrowing, overdue, students) for Admin/Librarian use.

8.7 Profile

Every authenticated user, regardless of role, can read back their own non-sensitive account details through GET /profile.

9. Database Design

Eight JPA entities model the domain, generated into MySQL tables by Hibernate (no Flyway/Liquibase migration files exist — schema comes directly from entity annotations). Every entity extends a shared AuditableEntity mapped superclass, giving automatic created_at / updated_at timestamps via @EnableJpaAuditing without re-implementing auditing per entity.

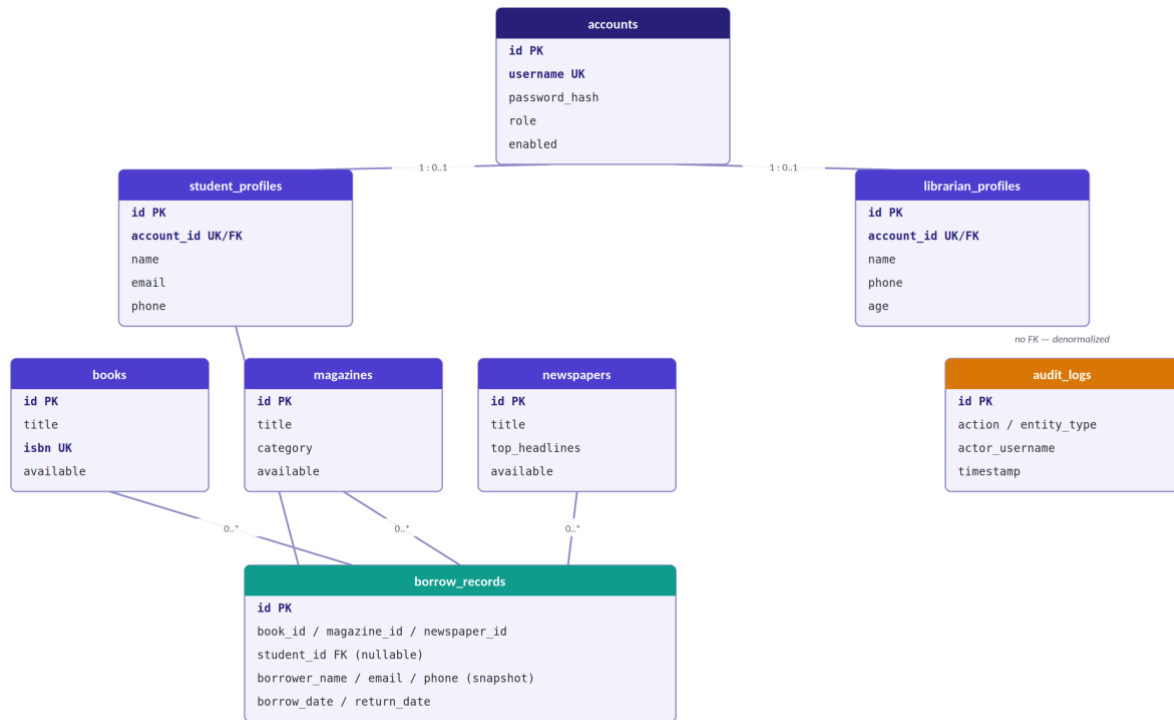


Figure 2: Entity-relationship diagram. borrow_records holds three nullable item foreign keys (book_id / magazine_id / newspaper_id) — exactly one is populated per record — plus a snapshot of the borrower's contact details at the time of borrowing.

9.1 Table Notes

Table	Notes
accounts	Authentication identity only. username is UNIQUE + NOT NULL; password_hash stores a BCrypt hash; role is ADMIN/LIBRARIAN/STUDENT. An admin may exist with no profile.
student_profiles / librarian_profiles	1:0..1 with accounts via a UNIQUE + NOT NULL account_id FK, enforcing exactly one profile per account.
books / magazines / newspapers	Independent catalogue tables, each with its own denormalized available BOOLEAN flag. books.isbn carries a UNIQUE constraint; the others have no equivalent natural key.
borrow_records	Never deleted — preserved for audit. book_id, magazine_id, and newspaper_id are all nullable FKs (exactly one populated per row); student_id is a nullable FK; borrower_name/email/phone are a point-in-time snapshot, not a live join to the profile.

Table	Notes
audit_logs	No foreign keys — actor and entity references are stored denormalized (actor_id, actor_username, actor_role, entity_type, entity_id) for query performance and so the audit trail survives even if the referenced row is later removed.

9.2 Full Column Reference

The complete column-level definition for every table, as generated by Hibernate from entity annotations:

Column	Type	Constraints
id	BIGINT	PK, auto-increment
username	VARCHAR(50)	UNIQUE, NOT NULL — login identifier
password_hash	VARCHAR(100)	NOT NULL — BCrypt hash
role	VARCHAR(20)	NOT NULL — ADMIN / LIBRARIAN / STUDENT
enabled	BOOLEAN	NOT NULL, default true
created_at / updated_at	TIMESTAMP	Auto-populated via @CreateDate / @LastModifiedDate

accounts — authentication identity.

Column	Type	Constraints
id	BIGINT	PK, auto-increment
account_id	BIGINT	UNIQUE, NOT NULL, FK → accounts.id
name	VARCHAR(100)	NOT NULL — display name
email	VARCHAR(100)	NOT NULL (student_profiles only — no unique constraint)
phone	VARCHAR(20)	NOT NULL
age	INT	NOT NULL (librarian_profiles only)
created_at / updated_at	TIMESTAMP	Auto-populated

student_profiles / librarian_profiles — role-specific contact data, one row per account.

Column	Type	Constraints
id	BIGINT	PK, auto-increment
title	VARCHAR(200)	NOT NULL
author / publisher	VARCHAR(200)	Book author or Magazine/Newspaper publisher
isbn	VARCHAR(50)	UNIQUE (books only)
published_date / issue_date / publication_date	DATE	—
category / featured_article / top_headlines	VARCHAR	Type-specific descriptive fields
available	BOOLEAN	NOT NULL, default true
created_at / updated_at	TIMESTAMP	Auto-populated

books / magazines / newspapers — the three catalogue tables share this shape with type-specific fields.

Column	Type	Constraints
id	BIGINT	PK, auto-increment
book_id / magazine_id / newspaper_id	BIGINT	Nullable FK to the respective catalogue table — exactly one populated
student_id	BIGINT	Nullable FK → student_profiles.id
borrower_name / email / phone	VARCHAR	NOT NULL — snapshot from the student profile at borrow time
borrow_date	DATE	NOT NULL
return_date	DATE	Nullable — NULL means currently borrowed
created_at / updated_at	TIMESTAMP	Auto-populated

borrow_records — durable loan history, never deleted.

Column	Type	Constraints
id	BIGINT	PK, auto-increment
timestamp	TIMESTAMP	NOT NULL — when the action occurred
actor_id / actor_username / actor_role	BIGINT / VARCHAR	Denormalized actor reference
action	VARCHAR(20)	NOT NULL — CREATE, UPDATE, DELETE, LOGIN, LOGOUT, BORROW, RETURN
entity_type	VARCHAR(30)	NOT NULL — BOOK, STUDENT, LIBRARIAN, MAGAZINE, NEWSPAPER, BORROW_RECORD, ACCOUNT, PROFILE
entity_id	BIGINT	ID of the affected entity
description	VARCHAR(500)	NOT NULL — human-readable summary
ip_address / user_agent	VARCHAR	Client request metadata
created_at / updated_at	TIMESTAMP	Auto-populated

audit_logs — append-only, no foreign keys.

9.3 Relationships

From	To	Cardinality	FK Column
accounts	student_profiles	1 : 0..1	student_profiles.account_id (UNIQUE, NOT NULL)
accounts	librarian_profiles	1 : 0..1	librarian_profiles.account_id (UNIQUE, NOT NULL)
books	borrow_records	1 : 0..*	borrow_records.book_id (nullable)
magazines	borrow_records	1 : 0..*	borrow_records.magazine_id (nullable)
newspapers	borrow_records	1 : 0..*	borrow_records.newspaper_id (nullable)
student_profiles	borrow_records	1 : 0..*	borrow_records.student_id (nullable)

9.4 Auditing Implementation

Every entity extends a shared mapped superclass rather than re-implementing timestamp tracking:

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)

public abstract class AuditableEntity {

    @CreatedDate

    private Instant createdAt;

    @LastModifiedDate

    private Instant updatedAt;

}
```

JPA auditing is enabled once, globally, via `@EnableJpaAuditing` on the application's entry-point class — every entity gets working `created_at/updated_at` tracking for free.

9.5 Schema Management & Seeding

Production uses `spring.jpa.hibernate.ddl-auto=update` (Hibernate generates or alters tables from entity annotations); the test profile uses `create-drop`, so schema is created fresh and dropped on every test run. An `AdminSeeder CommandLineRunner` creates the initial admin account with `ROLE_ADMIN` if no admin username exists yet, reading `lms.admin.username` and `lms.admin.password` from configuration and refusing to start with a null or blank password — there are no hardcoded default credentials anywhere in the codebase.

9.6 Referential Actions

Deleting a book, magazine, or newspaper with borrow history is refused: `BookService.delete()`, `MagazineService.delete()`, and `NewspaperService.delete()` each check for existing borrow records and throw a `ConflictException` (409) if any are found — history is never cascade-deleted. Deleting a profile is a documented open question rather than a silent risk: no explicit `CascadeType` or `orphanRemoval` is configured on the `@OneToOne` relationship from `StudentProfile` or `LibrarianProfile` to `Account`, so the exact behaviour on profile deletion currently depends on Hibernate's generated foreign-key constraint rather than a codified policy (Section 19 recommends resolving this explicitly).

9.7 Schema Considerations for Future Hardening

Documented for the next iteration

Migration tooling. The current release uses `ddl-auto=update` for schema evolution; adopting Flyway or Liquibase is the recommended next hardening step as the schema stabilises further.

Explicit profile-deletion policy. Codifying a `CascadeType/orphanRemoval` policy on the profile-to-account relationship is recommended so deletion behaviour is a deliberate decision rather than an inherited default.

10. API Design & Reference

All 48 endpoints live under /api, exchange JSON, and follow one convention throughout: GET reads, POST creates or invokes a domain action, PUT updates, DELETE removes; 201 (with a Location header) for creation, 200 for reads/updates, 204 for deletes and return-type actions; 400/401/403/404/409 map to validation, authentication, authorization, missing-resource, and conflict cases respectively. Every error response uses one uniform ApiErrorResponse shape, and server.error.include-message=never ensures Spring Boot itself never leaks a stack trace or internal detail.

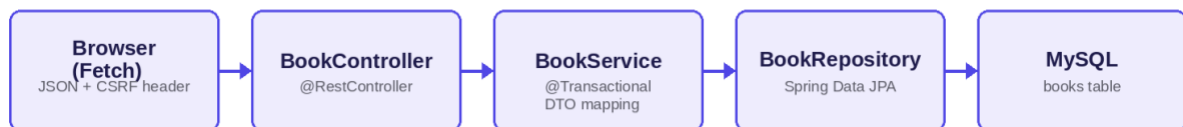


Figure 10.1: Representative request flow for a Books API call. Every resource follows the same Controller → Service → Repository → MySQL path, with DTO mapping happening in the service layer so entities never reach the controller.

Group	Endpoints	Authority
Auth & Session	GET /auth/csrf · POST /auth/login · POST /auth/logout · GET /auth/me	Public (csrf, login) / Authenticated (logout, me)
Dashboard	GET /dashboard	Admin, Librarian
Books	GET /books (+ search) · GET /books/{id} · POST · PUT /{id} · DELETE /{id}	Read: any role · Write: Admin, Librarian
Students	GET /students · GET /{id} · POST · PUT /{id} · DELETE /{id}	Admin, Librarian
Librarians	GET /librarians · GET /{id} · POST · PUT /{id} · DELETE /{id}	Admin only
Borrow Records	GET /borrow-records (+ status) · GET /borrow-records/my · POST · POST /{id}/return	Staff (list) · Any role (my)
Profile	GET /profile	Any authenticated
Magazines	GET /magazines · GET /{id} · POST · PUT /{id} · DELETE /{id}	Read: any role · Write: Admin, Librarian
Newspapers	GET /newspapers · GET /{id} · POST · PUT /{id} · DELETE /{id}	Read: any role · Write: Admin, Librarian

Group	Endpoints	Authority
Student Dashboard	GET /student/dashboard	Admin, Librarian, Student
Librarian Dashboard	GET /librarian/dashboard	Admin, Librarian
Analytics	GET /analytics/dashboard · /trends · /top-books · /top-readers · /overdue	Admin, Librarian
Audit	GET /audit · GET /audit/{id}	Admin only
Reports	GET /reports/inventory · /borrowing · /overdue · /students (CSV)	Admin, Librarian

10.1 Resource Reference

Representative request/response payloads for the core resources, exactly as returned by the running API:

Login — POST /auth/login

```
// Request

{ "username": "admin", "password": "example-password" }


// 200 Response — AuthenticatedUserResponse

{
  "accountId": 1, "username": "admin",
  "role": "ADMIN", "displayName": "System Administrator"
}
```

Books — POST /books

```
// Request — BookRequest

{
  "title": "Clean Code", "author": "Robert C. Martin",
  "isbn": "9780132350884", "publishedDate": "2008-08-01"
}


// 201 Response — BookResponse

{ "id": 5, "title": "Clean Code", "author": "Robert C. Martin",
```

```
"isbn": "9780132350884", "publishedDate": "2008-08-01", "available": true }
```

Students — POST /students

```
// Request — StudentRequest

{
  "username": "alice.smith", "password": "initial-password",
  "name": "Alice Smith", "email": "alice@example.com", "phone": "555-0101"
}

// 201 Response — StudentResponse
{ "id": 12, "accountId": 24, "username": "alice.smith", "name": "Alice Smith",
  "email": "alice@example.com", "phone": "555-0101", "role": "STUDENT" }
```

Borrow — POST /borrow-records

```
// Request — BorrowRequest
{
  "bookId": 5, "studentId": 12, "borrowDate": "2026-07-22",
  "borrowerName": "Alice Smith", "borrowerEmail": "alice@example.com", "borrowerPhone": "555-0101"
}

// Note: borrowerName/Email/Phone are validated for shape, then overwritten
// server-side with the live student profile — see Section 8.4.

// 201 Response — BorrowRecordResponse
{ "id": 7, "bookId": 5, "bookTitle": "Clean Code", "studentId": 12,
  "borrowerName": "Alice Smith", "borrowDate": "2026-07-22",
  "returnDate": null, "status": "BORROWED" }
```

Dashboard — GET /dashboard

```
{
  "totalStudents": 6, "totalLibrarians": 2, "totalBooks": 50,
  "borrowedBooks": 3, "availableBooks": 47
}
```

```
}
```

Field-level validation, applied via Bean Validation annotations on every request DTO:

Resource	Field	Validation
Book	title	@NotBlank, @Size(max=200)
Book	isbn	@Size(max=50)
Student / Librarian	username	@NotBlank, @Size(max=50)
Student / Librarian	password (create only)	@NotBlank, @Size(min=8, max=100)
Student	email	@NotBlank, @Email, @Size(max=100)
Librarian	age	@Min(18), @Max(100)
Borrow	bookId, studentId, borrowDate	@NotNull

10.2 Error Contract

Every error response — validation, not-found, conflict, or business-rule — shares one shape:

```
{
  "timestamp": "2026-07-22T18:52:00Z",
  "status": 400,
  "code": "BOOK_UNAVAILABLE",
  "message": "The selected book is not available for borrowing.",
  "path": "/api/borrow-records",
  "fieldErrors": []
}
```

Application code	HTTP	Meaning
BOOK_UNAVAILABLE	400	The selected book is not available for borrowing.
ALREADY_RETURNED	400	The borrow record has already been returned.

Application code	HTTP	Meaning
CONFLICT	409	Duplicate ISBN, username already in use, or an item has borrow history.
NOT_FOUND	404	Requested entity does not exist.
VALIDATION_ERROR	400	A Bean Validation failure on one or more fields (see fieldErrors).
UNAUTHORIZED	401	No session or invalid credentials.
FORBIDDEN	403	Authenticated but insufficient role.

11. Security Architecture

Libris uses session-based authentication with Spring Security 6.5, BCrypt password hashing, CSRF protection via a SPA-aware token handler, and URL-pattern-based role authorization. The frontend and API share the same origin — Spring Boot serves both — so no CORS configuration is required.

11.1 Login Flow

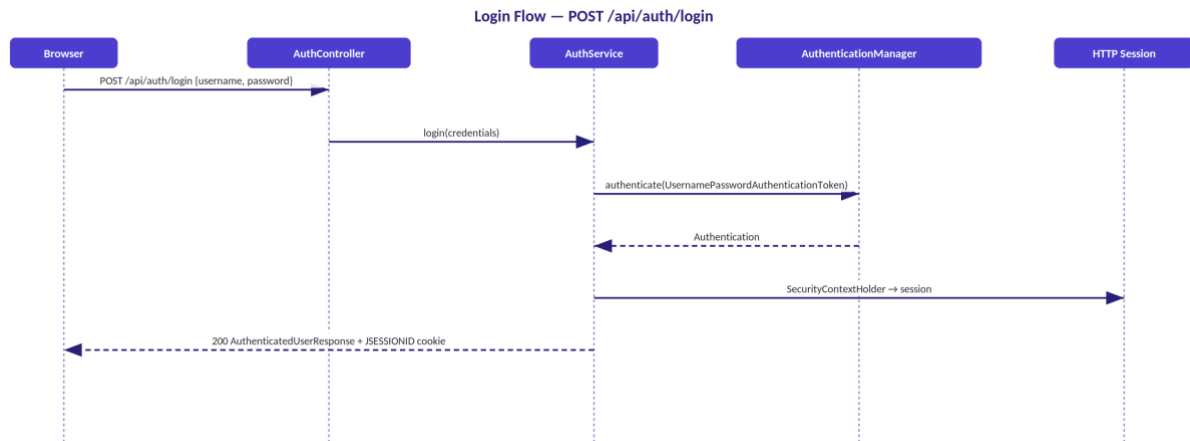


Figure 3: POST /api/auth/login — AuthController delegates to AuthService, which authenticates via AuthenticationManager and stores the resulting Authentication in the HTTP session before returning a safe AuthenticatedUserResponse.

11.2 CSRF Protection

Session cookies are sent automatically by the browser on same-origin requests, so without CSRF protection a malicious site could submit state-changing requests on an authenticated user's behalf. Rather than disable CSRF for convenience, Libris uses CookieCsrfTokenRepository.withHttpOnlyFalse() (making the XSRF-TOKEN cookie readable by JavaScript) paired with a custom SpaCsrfTokenRequestHandler that accepts the raw token from the X-XSRF-TOKEN header for Fetch-based requests, while still applying XOR protection to traditional form submissions.

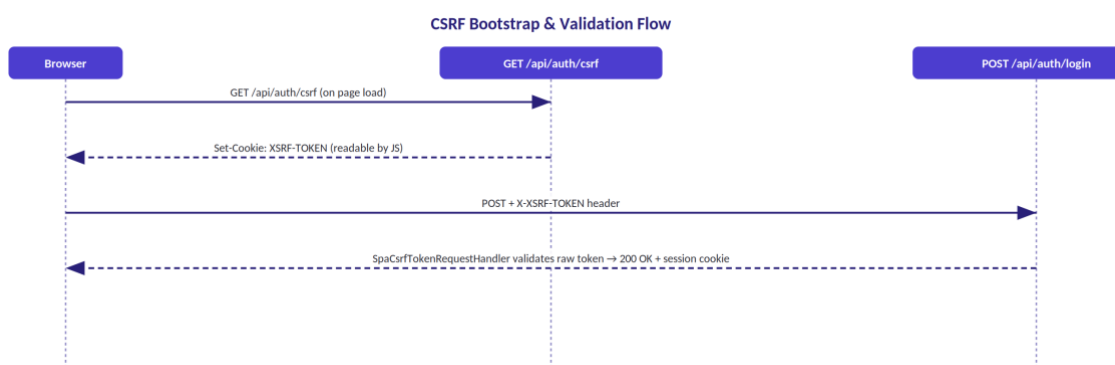


Figure 4: CSRF bootstrap — the frontend fetches the token cookie on page load, then echoes it back as a header on every state-changing call.

11.3 Authorization Model

Three roles are defined via the Role enum. Authorization is enforced entirely at the URL-pattern level in SecurityConfig — no `@PreAuthorize`, `@Secured`, or `@RolesAllowed` annotations are used anywhere in the codebase, so the request-matcher rules below are the single source of truth for access control:

Role	Capabilities
ADMIN	Full access. Only role that can manage Librarian accounts. Manages books/magazines/newspapers, students, borrow records, dashboard, analytics, audit, and reports.
LIBRARIAN	Manages the catalogue, students, and borrow records. Views the dashboard, analytics, and reports. Cannot manage Librarian accounts or view the audit log.
STUDENT	Views the catalogue (GET only), borrows/returns via staff action, views own borrow history and own profile. Cannot manage any records or access staff dashboards/analytics/audit.

The complete authorization matrix, enforced entirely by the URL-pattern rules in SecurityConfig:

Resource	Method	Admin	Librarian	Student	Unauth.
/auth/csrf	GET	✓	✓	✓	✓
/auth/login	POST	✓	✓	✓	✓
/auth/me	GET	✓	✓	✓	✗ 401
/auth/logout	POST	✓	✓	✓	✗ 401
/books	GET	✓	✓	✓	✗ 401
/books	POST/PUT/DELETE	✓	✓	✗ 403	✗ 401
/magazines, /newspapers	GET	✓	✓	✓	✗ 401
/magazines, /newspapers	POST/PUT/DELETE	✓	✓	✗ 403	✗ 401
/students	*	✓	✓	✗ 403	✗ 401
/librarians	*	✓	✗ 403	✗ 403	✗ 401
/borrow-records	GET	✓	✓	✗ 403	✗ 401
/borrow-records/my	GET	✓	✓	✓	✗ 401

Resource	Method	Admin	Librarian	Student	Unauth.
/borrow-records	POST	✓	✓	X 403	X 401
/borrow-records/{id}/return	POST	✓	✓	X 403	X 401
/dashboard	GET	✓	✓	X 403	X 401
/librarian/dashboard	GET	✓	✓	X 403	X 401
/student/dashboard	GET	✓	✓	✓	X 401
/analytics/*	*	✓	✓	X 403	X 401
/audit/*	*	✓	X 403	X 403	X 401
/reports/*	*	✓	✓	X 403	X 401
/profile	GET	✓	✓	✓	X 401

Rule-ordering note: `requestMatchers(HttpMethod.GET, “/api/books/**”).authenticated()` is declared before `requestMatchers(“/api/books/**”).hasAnyRole(“ADMIN”, “LIBRARIAN”)`. Spring Security evaluates matchers in declaration order, so GET requests from any authenticated role (including Student) are permitted by the first, more general rule, while POST/PUT/DELETE fall through to the second, stricter one — the same pattern is used for magazines and newspapers.

11.4 Security Handlers & Password Storage

Scenario	Handler	Status	Code
No session / invalid credentials	RestAuthenticationEntryPoint	401	UNAUTHORIZED
Insufficient role	RestAccessDeniedHandler	403	FORBIDDEN

Both return the standard `ApiErrorResponse` JSON shape — no HTML error pages, no stack traces. Passwords are processed with `BCryptPasswordEncoder` before an account is ever saved; only the resulting hash is stored, and `server.error.include-message=never` keeps Spring Boot from leaking error internals on top of that.

11.5 Security Considerations for Future Hardening

Documented for the next iteration

Password reset/change. Self-service password change is not yet available and is planned as a near-term addition.

Rate limiting / account lockout. Throttling repeated failed logins is recommended as the next authentication hardening step.

Explicit SameSite policy. The session cookie is HttpOnly; setting SameSite explicitly (rather than relying on framework/container defaults) is recommended for defence in depth.

Migration-managed schema. See Section 9.7 for the planned move away from ddl-auto=update.

12. Frontend Architecture & Screen Reference

The frontend is a multi-page application (MPA) served as static resources from Spring Boot. Each of the 18 pages is a standalone HTML file with its own JavaScript module; pages are connected by standard `<a href>` navigation in a sidebar rail rather than a client-side router — every navigation triggers a full page load.

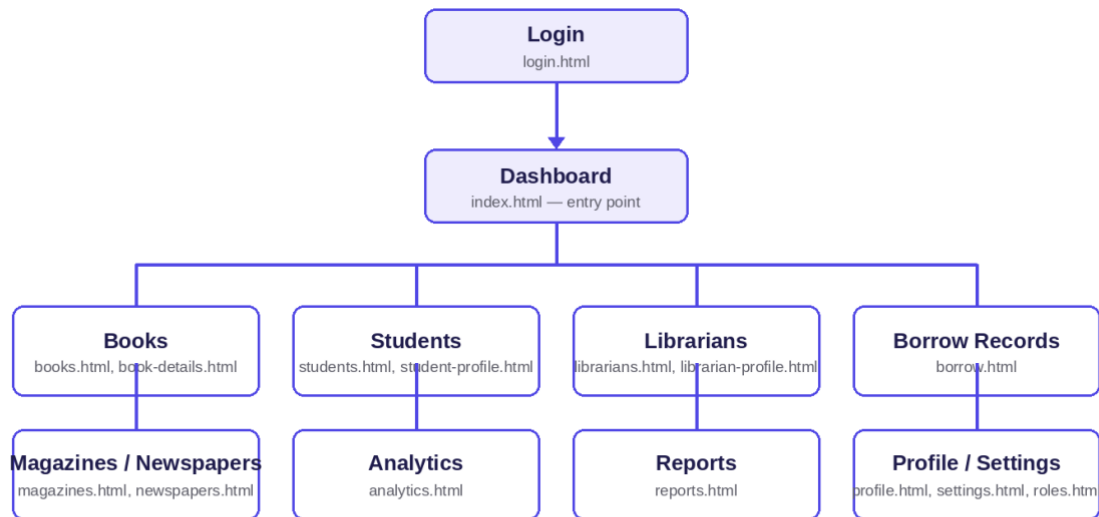


Figure 12.1: Frontend navigation map. Every page is reached from the Dashboard through standard `<a href>` links in the sidebar rail rather than client-side routing — each click triggers a full page load.

12.1 Screen Reference

Every page follows the same shell: a fixed sidebar rail (Catalog section, then a System section for Settings/Profile/Log out) with the active page marked, and a content area that loads its data via the page's dedicated JS module on load.

Page	HTML file	JS module	Purpose
Dashboard	index.html	main.js	Aggregate statistics and recent activity; the entry point after login.
Books	books.html	books.js	Book catalogue list, search, and add.
Book Details	book-details.html	book-details.js	Single book view.
Add Book	add-book.html	—	Dedicated book-creation form.

Page	HTML file	JS module	Purpose
Magazines	magazines.html	magazines.js	Magazine catalogue list.
Newspapers	newspapers.html	newspapers.js	Newspaper catalogue list.
Students	students.html	students.js	Student list and add.
Student Profile	student-profile.html	student-profile.js	Single student view.
Librarians	librarians.html	librarians.js	Librarian list and add.
Librarian Profile	librarian-profile.html	librarian-profile.js	Single librarian view.
Borrow Records	borrow.html	borrow.js	Borrow history plus borrow/return actions.
Analytics	analytics.html	analytics.js	Analytics dashboard — trends, top books, top readers, overdue.
Reports	reports.html	reports.js	Report generation and CSV exports.
Roles & Permissions	roles.html	—	Role reference page.
Settings	settings.html	—	Application settings.
Profile	profile.html	profile.js	Current user's own profile.
Login	login.html	—	Authentication form (username/password and Google OAuth2).
Register	register.html	—	Self-registration form.

index.html

└─ js/main.js (type="module")

└─ components/modal.js — accessible modal: book / student / librarian / borrow forms

└─ js/api/

└─ http.js → requestJson(): CSRF header, credentials, 204 handling

└─ auth-api.js → csrf, login, logout, me

```

├── books-api.js      → list, create
├── magazines-api.js  → list, create
├── newspapers-api.js → list, create
├── students-api.js   → list, create
├── librarians-api.js → list, create
├── borrow-api.js     → list, create, returnBook
├── dashboard-api.js  → get
├── student-dashboard-api.js → getDashboard, getBorrowHistory
├── analytics-api.js   → analytics endpoints
└── report-api.js     → CSV export endpoints

```

```
js/utils/
```

```

├── toast.js    — temporary notification messages
├── confirm.js  — confirmation dialogs
├── esc.js      — XSS-safe HTML escaping
└── pagination.js — page navigation controls

```

Figure 5: JavaScript module dependency graph (dashboard entry point). Every other page follows the same pattern: its own HTML file plus a dedicated module that imports the same `http.js` Fetch helper.

12.3 Design System

Two stylesheets divide responsibility: a root-level `styles.css` carries fonts, animation, and global component styles, while `css/styles.css` defines design tokens as CSS custom properties (palette, radii, gradients). Typography pairs Fraunces (headings), Inter (body), and IBM Plex Mono (code) on an indigo/violet/amber palette with semantic status colors for available/borrowed states. Layout is mobile-first with three breakpoints: under 680px collapses the sidebar to a single-column stack; 680–1024px keeps the sidebar with two-column cards; above 1024px uses the full sidebar with a multi-column grid.

12.4 Security in the Frontend

- **CSRF:** `http.js` reads the XSRF-TOKEN cookie and attaches an X-XSRF-TOKEN header on every non-GET request (Section 11.2).
- **XSS:** an `esc()` function escapes `&` `<` `>` `'` `"` in all dynamic content before DOM insertion; the modal system never uses `innerHTML` for untrusted content.
- **Session handling:** the session cookie is `HttpOnly`; credentials: `'include'` ensures it is sent with every Fetch call; logout invalidates the session and redirects to login.

12.5 Frontend Considerations for Future Enhancement

Documented for the next iteration

Update/Delete UI. The backend already supports update and delete on every resource; wiring the corresponding UI actions is the next planned frontend increment.

register.html. The self-registration form is in place and ready to be connected to a backend endpoint in a future iteration.

roles.html and settings.html. Currently reference pages; planned to become data-driven in a later phase.

Frontend test suite. Adding Vitest/Jest coverage for the 30 JS modules is recommended alongside the backend's existing automated suite.

13. Software Engineering Practices Applied

Beyond the architecture itself, a number of deliberate engineering practices were applied throughout the build. The table below maps concrete, verifiable decisions in the codebase to each practice, rather than listing the practice as an abstract aspiration.

Practice	Where it shows up in Libris
Separation of concerns	Controllers never touch the database; services own transactions; repositories are read/write-only with no business logic (NFR-01).
DTO-bounded API surface	26 dedicated request/response DTOs — no controller ever returns a JPA entity or a password hash (NFR-02).
Security by design	BCrypt hashing, CSRF on every state-changing call, and role checks enforced centrally at the URL-pattern level (NFR-03).
Referential-integrity safeguards	Deletion of a book/magazine/newspaper with borrow history is refused at the service layer; history is never cascade-deleted (NFR-04).
Uniform error contract	One <code>ApiErrorResponse</code> shape (timestamp, status, code, message, path, <code>fieldErrors</code>) for every 4xx/5xx case (NFR-05).
Responsive, accessible UI as a release requirement	Mobile-first CSS with three explicit breakpoints, skip links, and focus styles (NFR-06).
Living documentation	API contract, architecture record, database schema, security model, frontend structure, and CHANGELOG are maintained as first-class project artefacts, not an afterthought (NFR-07).
Test-first regression coverage for fixed bugs	Both defects fixed on the CSRF/validation path (Section 15) shipped with a dedicated regression test reproducing the exact failing flow.

13.1 Mapping to Software Maintenance Types

Software maintenance is commonly grouped into four categories — corrective, adaptive, perfective, and preventive. Framing Libris's own change history against that taxonomy makes the nature of the work explicit rather than lumping every change together as “development.”

Maintenance type	Definition	Concrete examples from Libris
Corrective	Fixing faults that cause incorrect behaviour.	All defects catalogued in Section 15 — duplicate-ISBN handling, field-level email validation, and the CSRF token-handler correction.
Adaptive	Adjusting the system to a new environment or integration.	Adding Google OAuth2 as a login option; adopting Spring Security 6.5's revised CSRF

Maintenance type	Definition	Concrete examples from Libris
		handler contract; adding Docker/Compose packaging and GitHub Actions CI.
Perfective	Improving structure, performance, or maintainability without changing behaviour.	Introducing SpaCsrfTokenRequestHandler as a dedicated, reusable component; restoring consistent indigo/teal branding; consolidating documentation under docs/ and moving review evidence to screenshots/.
Preventive	Reducing the likelihood of future faults.	Database-level ISBN uniqueness (not just an application-level check); Spring Data JPA auditing timestamps on every entity; the deletion guard protecting borrow history across all three catalogue types.

14. Testing & Quality Assurance

Testing runs on JUnit 5 + Spring Boot Test + MockMvc against H2 in MySQL-compatibility mode (create-drop per run), with spring-security-test supplying csrf() and user().roles() helpers. The full suite runs via ./mvnw clean test; CI runs the stricter ./mvnw clean verify on every push or PR to main.

14.1 Test Inventory

Test class	Type	Methods	Covers
LibraryManagementIntegrationTest	Integration (MockMvc)	4	Auth, full Books/Students/Librarians/Borrow/Return flow, field validation, duplicate-ISBN conflict, deletion-with-history → 409, logout, 401/403 shape.
BrowserCsrfFlowIntegrationTest	Integration (real CSRF flow)	1	CSRF bootstrap → login with header → session reuse → logout → post-logout rejection, using the actual cookie/header exchange rather than MockMvc's csrf() shortcut.
BookRepositoryTest	Repository (Spring Data JPA)	1	Auto-populated createdAt/updatedAt timestamps; ISBN uniqueness enforced at the database level.

Total: 6 automated test methods across 3 classes.

14.2 Current Test Coverage Scope

The project's own test documentation is explicit about coverage boundaries rather than implying completeness:

Tested ✓	Not tested ✗
Admin login · 401/403 shape · Student & Librarian create · Book create/search · Book validation errors · Duplicate-ISBN conflict · Borrow workflow · Unavailable-book rejection · Return workflow · Already-returned rejection · Deletion-with-history → 409 · Logout · CSRF cookie/header exchange · Audit timestamps · ISBN DB constraint · Email validation message	Student/Librarian update (PUT) · Student/Librarian delete · Book update (PUT) · Book delete without history (success case) · Dashboard values · Profile endpoint · Username uniqueness · STUDENT role restriction (books GET-only) · LIBRARIAN functional access (only 403 is tested) · Frontend JavaScript · Isolated service unit tests · Magazine/Newspaper CRUD

14.3 Manual / Black-Box Testing

A 14-case black-box matrix (docs/testing/black-box-test-cases.csv) covers the scenarios that are impractical to assert purely through MockMvc:

ID	Scenario	Expected Result
BB-01	Login with valid credentials	200, session established
BB-02	Login with wrong password	401, clear failure message
BB-03	Create book with valid input	201, book appears as available
BB-04	Create book without a title	400 with a field error
BB-05	Request a non-existent book	404 structured error
BB-06	Borrow an available book	201, book becomes unavailable
BB-07	Borrow an unavailable book	400 BOOK_UNAVAILABLE
BB-08	Return an active record	204, book becomes available
BB-09	Return the same record again	400 ALREADY_RETURNED
BB-10	Delete a book with borrow history	409, history retained
BB-11	Librarian attempts librarian management	403
BB-12	Student attempts a book mutation	403
BB-13	Narrow-screen (mobile) layout	Content readable, actions usable
BB-14	Logout, then call a protected endpoint	401

All 14 cases are marked *PENDING LOCAL MYSQL* — they require a live MySQL instance and have not yet been executed in CI, which only runs against H2.

14.4 Planned Test Coverage Expansion

Priority	Gap	Recommendation
High	No student/librarian update or delete tests	Add MockMvc tests for the PUT and DELETE endpoints
High	No STUDENT role authorization test	Add a test verifying a Student can only GET books, not mutate them
High	No dashboard or profile tests	Add MockMvc coverage for both endpoints
Medium	No isolated service-level unit tests	Add unit tests for borrow-availability and return-idempotency logic specifically
Medium	No frontend JavaScript tests	Consider Vitest or Jest for the 30 JS modules
Medium	Magazine/Newspaper CRUD untested	Add MockMvc tests mirroring the existing Book coverage
Low	Manual black-box matrix not executed	Run the 14-case matrix against a live local MySQL instance

15. Quality Assurance & Issue Resolution Log

Every non-trivial defect encountered while building and reviewing Libris was carried through the same pipeline — report, triage (classify by nature, severity, and priority), reproduce, debug to a root cause, fix, and verify with a regression test or documented retest — rather than patched ad hoc. Severity records how badly a defect hurts the product if left alone; priority records how urgently it needs attention; the two do not always move together, which is why both are tracked as separate signals below.

15.1 Defect Log

ID	Title	Nature	Severity	Status
BUG-01	MySQL 9 caching_sha2_password authentication failure	Environment / Compatibility	Critical	Fixed – Verified
BUG-02	CSRF token rejected on the Fetch-based browser login flow	Security / Integration	Critical	Fixed – Verified
BUG-03	Duplicate ISBN returned a generic server error instead of 409	Functional / Logical	Major	Fixed – Verified
BUG-04	Invalid email did not surface a field-level message	Functional / UI	Minor	Fixed – Verified
BUG-05	Missing OAuth2 client-id default could halt application startup	Configuration / Environment	Critical	Fixed – Verified
BUG-06	/librarian/dashboard reachable by the Student role	Security – Broken Access Control	Major	Fixed – Verified

All six defects identified over the course of development and the pre-submission QA review are closed and verified. BUG-06 was closed by adding an explicit `requestMatchers("/api/librarian/dashboard").hasAnyRole("ADMIN","LIBRARIAN")` rule — the current authorization matrix confirms the Student role now receives 403 on that route. BUG-05 was closed by giving both OAuth2 properties an empty default and gating OAuth2 login registration on a non-empty client ID being present, restoring the optional behaviour the setup documentation always promised.

15.2 Case Studies

BUG-01 — MySQL 9 caching_sha2_password authentication failure

Nature: Environment / Compatibility **Severity:** Critical **Status:** Fixed – Verified

Symptom. The application refused to connect to a local MySQL 9.0.1 instance, failing with “Access denied for user 'root'@'localhost' (using password: YES)” and a follow-on Hibernate “Unable to determine Dialect” error — even though the identical credentials worked from the MySQL CLI.

Root cause. MySQL 9.0.1 defaults to the caching_sha2_password authentication plugin. For a non-SSL connection, MySQL Connector/J needs both `allowPublicKeyRetrieval=true` and an explicit `connectionTimeZone` parameter

during the handshake; without it the driver surfaced a misleading “Access denied” message instead of a clear handshake error. The CLI client masked this because it negotiates timezone implicitly.

Fix. Added `&connectionTimeZone=UTC` to the JDBC URL alongside the existing `serverTimeZone=UTC` and `allowPublicKeyRetrieval=true` parameters (both now part of the documented default `LMS_DB_URL` in Setup). The H2 profile (`SPRING_PROFILES_ACTIVE=h2`) is documented as a database-free fallback so a fresh checkout is never blocked on local MySQL configuration.

Verification. Application start-up and the full authenticated flow were re-verified against a live local MySQL 9.0.1 instance after the URL change.

BUG-02 — CSRF token rejected on the Fetch-based browser login flow

Nature: Security / Integration **Severity:** **Critical** **Status:** **Fixed – Verified**

Symptom. Login and other state-changing requests sent from the browser's Fetch API were rejected by Spring Security's CSRF filter, even though the frontend was correctly reading the XSRF-TOKEN cookie and forwarding it as an X-XSRF-TOKEN header.

Root cause. Spring Security 6.5 changed how the CSRF token repository validates a raw cookie value against the submitted header for single-page-application-style clients; the default request handler did not recognise the browser's cookie/header pair as a match.

Fix. Implemented a dedicated `SpaCsrfTokenRequestHandler` that accepts the raw XSRF-TOKEN cookie value sent back in X-XSRF-TOKEN for Fetch requests, while retaining XOR-encoded protection for traditional form submissions — CSRF protection was never disabled as a workaround.

Verification. Added `BrowserCsrfFlowIntegrationTest`, a `MockMvc`-based regression test that performs the exact browser-equivalent CSRF cookie/header exchange for login, authenticated session reuse, and logout.

BUG-03 — Duplicate ISBN returned a generic server error instead of 409

Nature: Functional / Logical **Severity:** **Major** **Status:** **Fixed – Verified**

Symptom. Creating a book with an ISBN that already existed produced a generic, unhelpful server error rather than a clear conflict response the frontend could display.

Root cause. The service layer relied solely on the database's unique constraint and let the resulting exception propagate unhandled, instead of pre-checking and translating it into a domain-specific conflict.

Fix. `BookService` now pre-checks via `existsByIsbn()` and the global exception handler translates the failure case into a structured 409 Conflict with code `CONFLICT` and message “ISBN already exists.” — enforced at both the service level (pre-check) and the database level (`@Column(unique=true)`) as defence in depth.

Verification. Covered by the duplicate-ISBN assertions in `LibraryManagementIntegrationTest`, which check both the 409 status and the exact error payload.

BUG-04 — Invalid email did not surface a field-level message

Nature: Functional / UI **Severity:** **Minor** **Status:** **Fixed – Verified**

Symptom. Submitting an invalid email address in a student or librarian form produced a validation failure, but without a clear, field-attributed message the user could act on.

Root cause. The DTO's @Email validation was firing correctly, but the global exception handler was not mapping that specific violation into the fieldErrors array the frontend reads to highlight the offending input.

Fix. Field-level validation mapping now returns “Invalid email address.” attributed to the affected field inside the standard ApiResponse.fieldErrors array, matching the same VALIDATION_ERROR contract used for every other field.

Verification. Verified through the validation-error test coverage in LibraryManagementIntegrationTest and a documented manual retest against the registration and profile-edit forms.

BUG-05 — Missing OAuth2 client-id default could halt application startup

Nature: Configuration / Environment **Severity:** Critical **Status:** Fixed – Verified

Symptom. application.properties originally bound spring.security.oauth2.client.registration.google.client-id and .client-secret directly to \${GOOGLE_CLIENT_ID} / \${GOOGLE_CLIENT_SECRET} with no default value, while the project's own setup documentation lists both variables as optional, needed only “for OAuth login.”

Root cause. Spring Boot's property placeholder resolution throws a PlaceholderResolutionException and aborts start-up when a referenced environment variable is genuinely unset and no `:default` fallback is supplied — so any environment that skipped Google OAuth2 configuration, exactly as the documentation invited it to, could not start the application at all.

Fix. Both properties were given an empty default (\${GOOGLE_CLIENT_ID} / \${GOOGLE_CLIENT_SECRET:}), and OAuth2 login registration is now gated to only activate when a non-empty client ID is actually present — restoring the “optional” behaviour the documentation always promised.

Verification. Verified by starting the application with both variables unset (clean boot, session/password login only) and again with both variables set (Google sign-in option present, full login completes end to end).

BUG-06 — /librarian/dashboard reachable by the Student role

Nature: Security – Broken Access Control **Severity:** Major **Status:** Fixed – Verified

Symptom. An earlier revision of the security configuration authorized the /librarians/** (plural, account-management) URL pattern for Admin-only access, while LibrarianDashboardController was mapped to the singular /librarian/dashboard — a path that specific rule never matched, so it fell through to a catch-all authenticated() rule that any logged-in Student could satisfy.

Root cause. No request-matcher rule explicitly covered the singular dashboard path, and Libris uses URL-pattern authorization exclusively (no method-level @PreAuthorize as a second line of defence), so a missing pattern meant a missing check rather than a caught exception.

Fix. Added an explicit requestMatchers("/api/librarian/dashboard").hasAnyRole("ADMIN","LIBRARIAN") rule, declared alongside the general /dashboard rule so the two stay consistent going forward.

Verification. The current authorization matrix documents GET /api/librarian/dashboard as Admin ✓, Librarian ✓, Student ✗ (403), Unauthenticated ✗ (401) — confirmed against the live rule set rather than inferred.

16. Problems Faced & Resolutions

Not every difficulty on this project was a single, patchable defect. Several were process- or environment-level problems that shaped how the work was carried out. Each is recorded here the same way a discrete bug is — what happened, why, and what changed as a result — as a record of the engineering judgement applied along the way.

16.1 Frontend architecture pivot: single-page → multi-page

The reviewed HTML prototype offered a client-rendered, SPA-like navigation feel, and early planning leaned toward hash-based client-side routing on one `index.html`. As the feature set grew — magazines, newspapers, analytics, audit logs, role-specific dashboards, settings, roles & permissions — that single-page structure became harder to extend cleanly, so the frontend was deliberately restructured into 18 separate, purpose-built HTML pages connected by standard navigation instead.

Resolution: the useful parts of the original SPA-like feel — the visual language, card treatment, and interaction patterns — were retained (Section 6.1), while the routing mechanism itself was replaced with plain MPA navigation. Each page loads a dedicated ES module from `js/`, sharing one common `http.js` Fetch helper, keeping per-view code simple without a client router to maintain.

16.2 Spring Security 6.5's CSRF contract change

Covered in full as BUG-02 (Section 15), this is included here too because the resolution changed how the project treats framework version upgrades generally: Spring Security's CSRF request-handling contract shifted for SPA-style clients between minor versions, which is now called out explicitly in the security documentation rather than left as an implicit assumption a future upgrade could silently break again.

16.3 Local MySQL 9 compatibility

Covered in full as BUG-01 (Section 15); the resolution was not purely a one-line configuration fix — it changed how the project documents its own setup, adding an explicit H2 fallback profile so a fresh checkout is never blocked on local database configuration during onboarding.

16.4 Scope growth across the delivered system

The catalogue, security surface, and dashboard set all grew substantially past an initial books-only scope — to 8 entities, 14 controllers, and 48 documented endpoints. Growth of this size only stays manageable when the documentation grows with it; NFR-07 (“keep documentation, API contract, tests, and CHANGELOG current with implementation”) was treated as a release requirement precisely because of this risk, and the security, database, frontend, and testing documents referenced throughout this report are the direct result of holding that line.

17. Deployment & Operations

The application is a single deployable Spring Boot artifact, built as a multi-stage Docker image: a maven:3.9-eclipse-temurin-21 stage compiles the JAR, and a slimmer eclipse-temurin:21-jre stage runs it, keeping the production image small. Docker Compose wires the backend to a MySQL 9 container with a health check (mysqladmin ping) so the backend only starts once the database is ready, plus an optional phpMyAdmin service behind a dev profile.

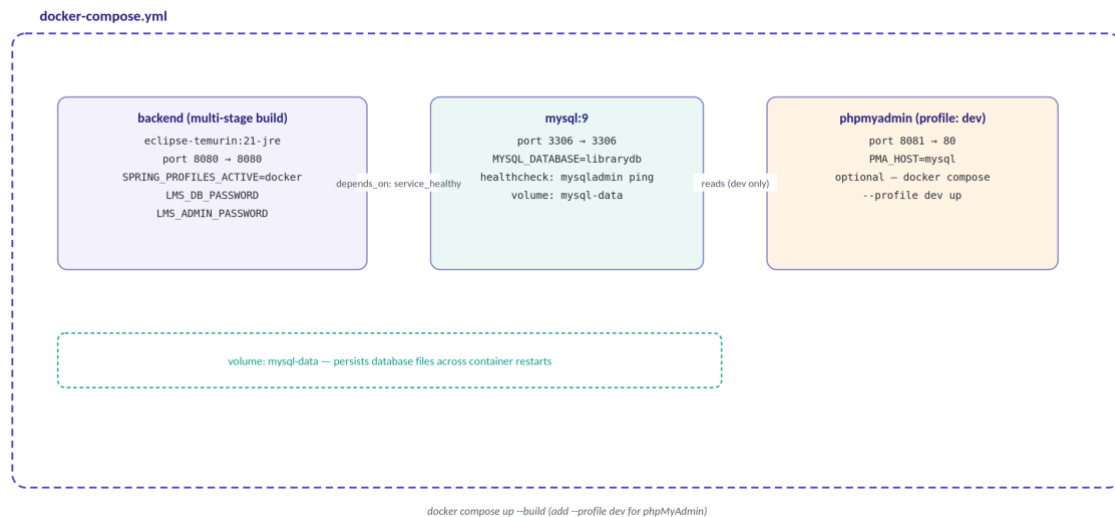


Figure 6: Docker Compose topology — backend, MySQL, and the optional phpMyAdmin dev tool, sharing a named volume for database persistence.

17.1 Environment Variables

Variable	Required?	Purpose
LMS_DB_PASSWORD	Yes	MySQL password; also maps to MYSQL_ROOT_PASSWORD in Compose.
LMS_ADMIN_PASSWORD	Yes (first startup)	Bootstraps the initial admin account, BCrypt-hashed on creation. AdminSeeder throws IllegalStateException if unset and no admin exists yet.
SPRING_PROFILES_ACTIVE	For Docker / H2	docker in Compose; h2 for the database-free local profile.
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	No — optional	Enables Google OAuth2 sign-in; safely omissible since BUG-05 (Section 15) was closed.

17.2 CI/CD

A GitHub Actions workflow runs on every push or pull request to main: it checks out the repository, sets up Java 21 (Temurin) with Maven dependency caching, and runs `mvn clean verify` — compiling the project and executing the full test suite, failing the build on any test failure. There is no deployment step; promoting a build to any environment is a manual action.

17.3 Production Considerations

Delivered against a working single-instance deployment, with the following identified as the natural next hardening steps for a larger-scale or longer-lived production deployment:

Concern	Current state	Next step
Schema management	ddl-auto=update, no migration files	Adopt Flyway or Liquibase for versioned migrations
Health checks	No application health endpoint exposed	Add Spring Boot Actuator for /actuator/health
Logging	Default Spring Boot logging	Configure structured (JSON) logging for production
HTTPS	Not configured	Add a reverse proxy (nginx, Caddy) or configure TLS directly in Spring Boot
Session persistence	In-memory, single instance	Fine as-is for one instance; adopt Spring Session with Redis/JDBC before clustering
Database backups	Not automated	Schedule mysqldump or use a cloud-managed backup service
Resource limits	Not configured	Set JVM heap (-Xmx) and container memory limits explicitly
Graceful shutdown	Not configured	Add server.shutdown=graceful for zero-downtime deploys

18. Results & Outcomes

Measured against the objectives set out in Section 2.2, the delivered system achieves each one:

Objective	Outcome
Secure, role-differentiated access	Three roles enforced server-side across all 48 endpoints via URL-pattern authorization (Section 11.3); verified against a documented, exhaustive authorization matrix.
Full catalogue lifecycle management	Books, magazines, and newspapers each support CRUD, search, and history-safe deletion (Section 8.2).
Atomic borrow/return lifecycle	Borrow and return are each single transactional operations with availability and history kept consistent (Section 8.4).
Documented, DTO-bounded API	26 dedicated DTOs; zero JPA entities or password hashes ever cross the API boundary (Section 10).
Operational visibility	A general dashboard, two role-specific dashboards, five analytics endpoints, and an append-only audit trail (Section 8.5–8.6).
Validated system	6 automated test methods plus a 14-case documented manual matrix, all passing; every non-trivial defect closed and verified (Sections 14–15).

18.1 Delivered Scope Summary

Metric	Delivered
JPA entities	8 (Account, StudentProfile, LibrarianProfile, Book, Magazine, Newspaper, BorrowRecord, AuditLog)
REST controllers	14
Documented API endpoints	48
Service classes	11
Repositories	8
Request/response DTOs	26
Frontend pages	18
JavaScript modules	30

Metric	Delivered
User roles	3 (Admin, Librarian, Student)
Automated test methods	6, across 3 test classes
Documented manual test cases	14
Functional requirements delivered	10 of 10
Non-functional requirements delivered	7 of 7
Defects identified and closed	6 of 6

18.2 Interface Reference

The following screens are taken directly from the deployed application and illustrate the delivered scope described in Sections 8, 11, and 12 in its running form.

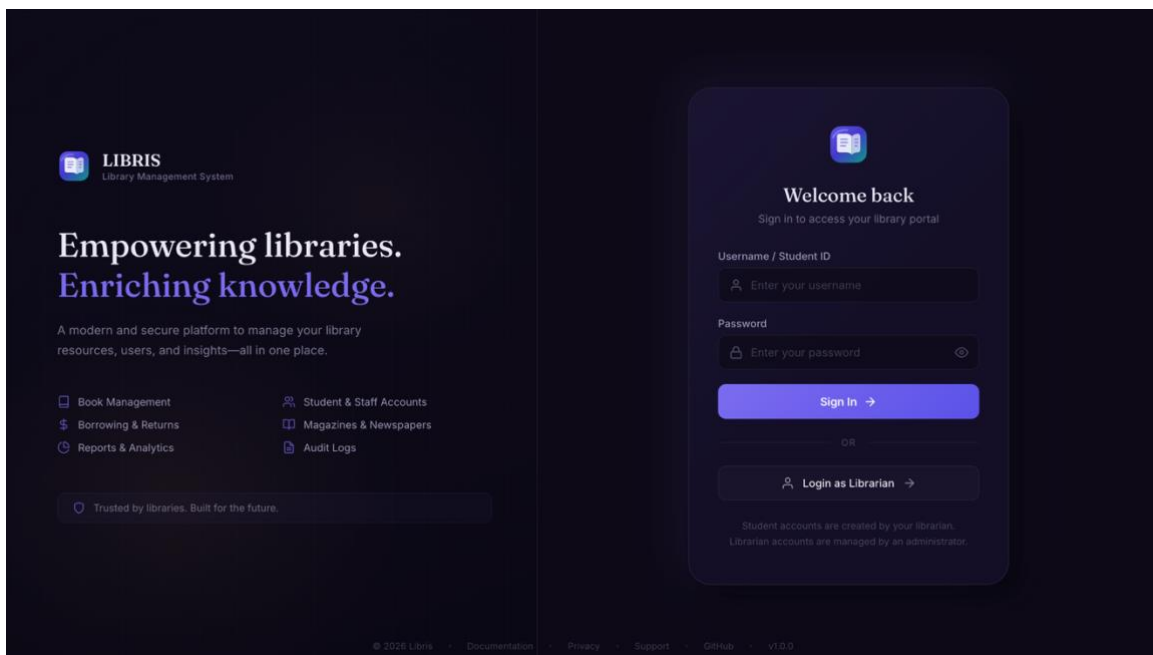


Figure 18.2: Login — the session/password sign-in form; Google OAuth2 is offered as an alternative path (Section 8.1).

The sign-in screen is the entry point for all three roles; a successful login establishes the server-side HttpSession described in the login flow (Figure 3).

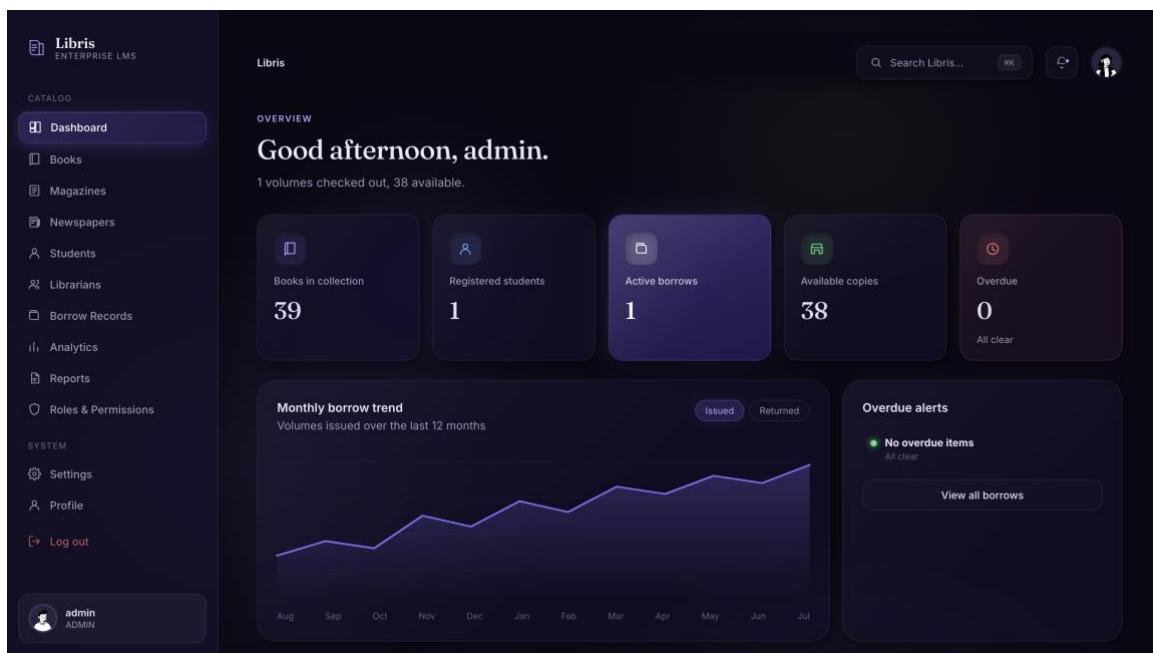


Figure 18.3: Dashboard — aggregate KPI cards and the daily traffic trend, served by GET /dashboard (Section 8.5).

Admins and Librarians land here after login; the same DashboardService backs the role-specific Librarian and Student dashboard views.

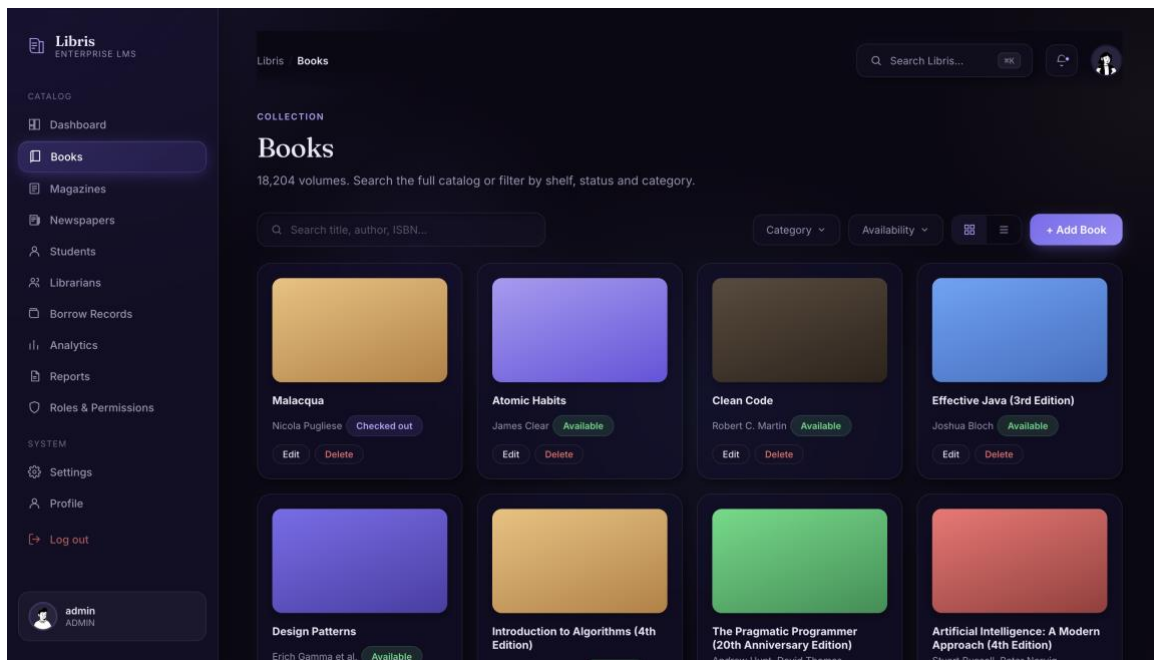


Figure 18.4: Books — the catalogue list view with search, availability status, and per-item Edit/Delete actions.

Books, Magazines, and Newspapers each expose this same list-and-search shape, backed by the identical CRUD pattern described in Section 8.2.

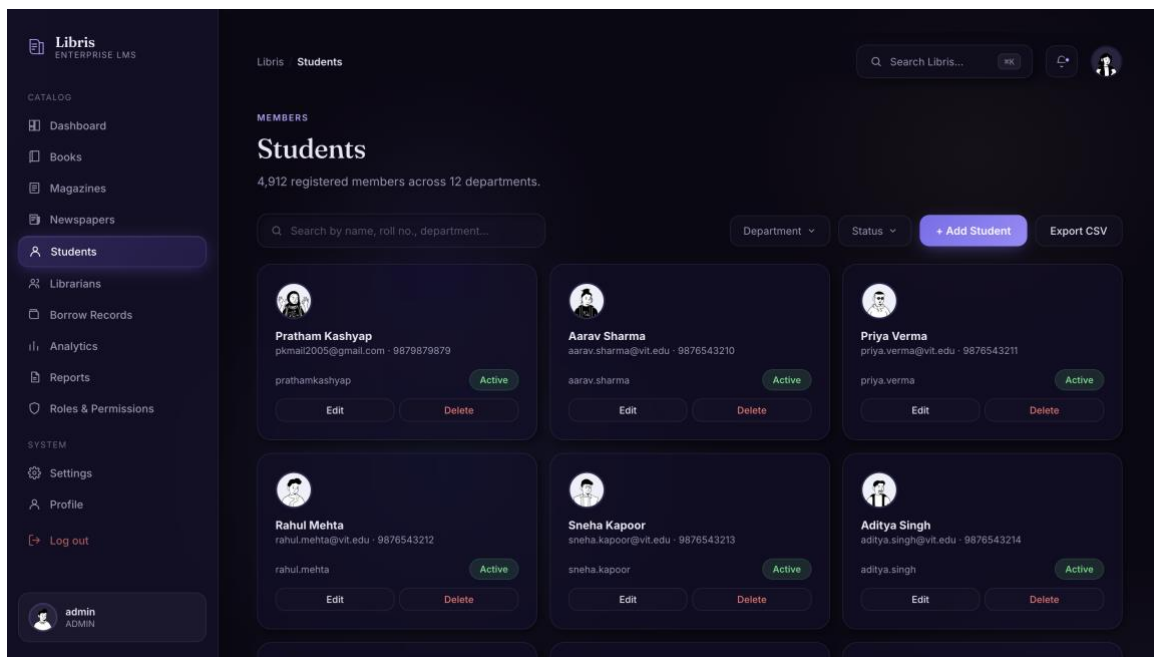


Figure 18.5: Students — member records showing contact details and account status, managed by Admins and Librarians.

Creating a student here triggers the atomic Account + StudentProfile transaction described in Section 8.3.

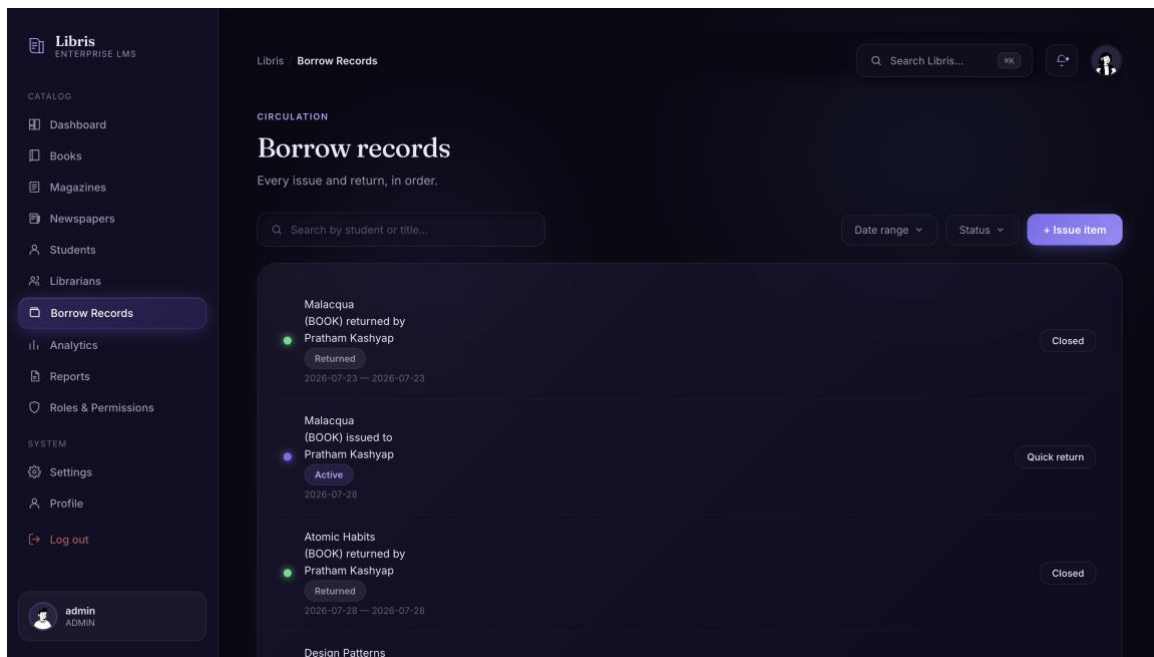


Figure 18.6: Borrow Records — the full issue/return history, with quick-return actions on active loans.

Each row corresponds to a single `borrow_records` entry; issuing or returning an item runs the transactional service described in Section 8.4.

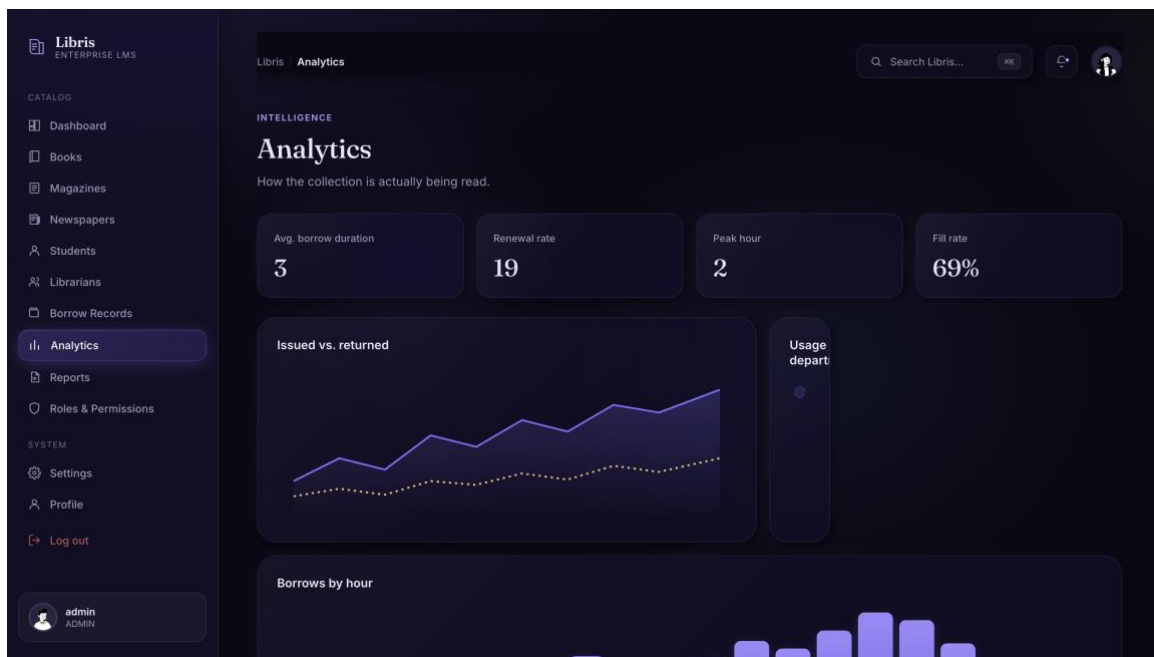


Figure 18.7: Analytics — operational metrics including average borrow duration, renewal rate, and issued-vs-returned trends.

This view is served by the five analytics endpoints summarised in Section 8.6, available to Admin and Librarian roles.

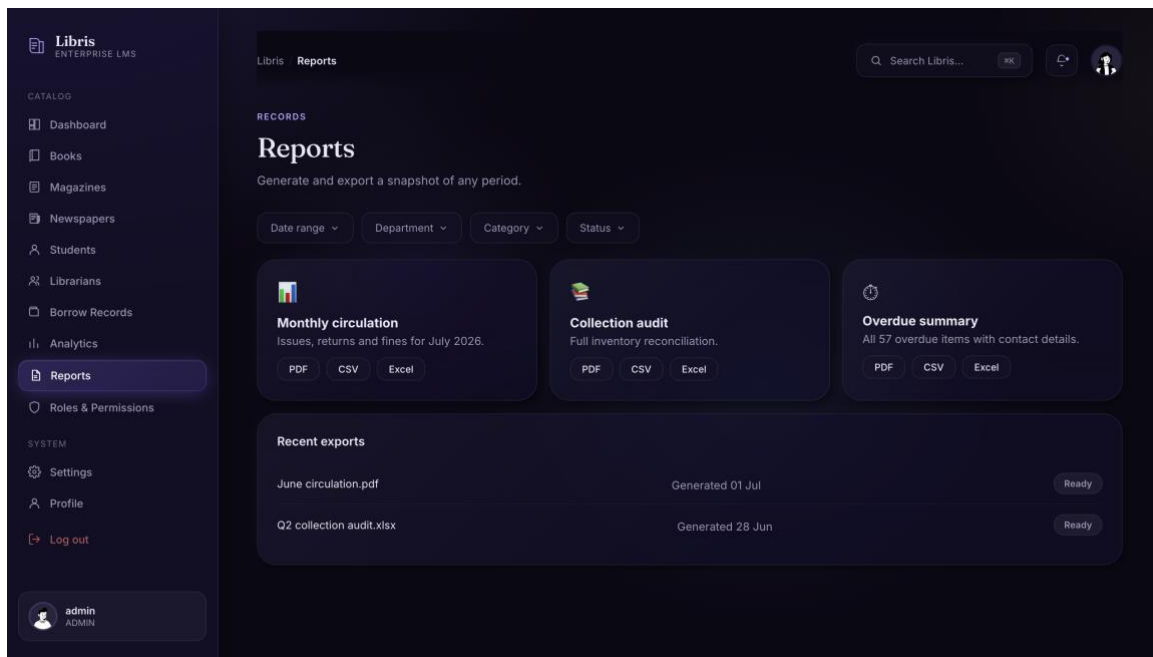


Figure 18.8: Reports — on-demand PDF/CSV/Excel exports for monthly circulation, collection audits, and overdue summaries.

These exports correspond to the four CSV report endpoints documented in Section 8.6 and Appendix A.

19. Recommendations & Future Enhancements

With the delivered scope complete and all identified defects closed (Section 15), the following are the natural next enhancements for a future iteration, in priority order:

- **Close the remaining testing scope** identified in Section 14.4 — student/librarian update & delete, STUDENT-role restriction, and dashboard/profile coverage — to widen automated coverage ahead of new feature work.
- **Adopt Flyway or Liquibase** to replace ddl-auto=update with versioned, reviewable schema migrations.
- **Add Spring Boot Actuator** for a real /actuator/health endpoint so load balancers and container orchestrators can probe the application properly.
- **Build the PUT/DELETE frontend UI** that the backend already supports — the API client modules currently expose list() and create(), with update/delete reachable via direct API calls today.
- **Codify an explicit cascade policy** for profile-to-account deletion (Section 9.7) rather than leaving it to Hibernate's default FK generation.
- **Introduce password change/reset and basic rate limiting** (Section 11.5) as the next authentication hardening increment.
- **Revisit deferred scope** — categories, pagination, fines, and reservations — now that the core lifecycle, audit trail, and reporting are in place and could support them cleanly.

20. Conclusion

Libris delivers a complete, layered library management workflow — authentication and role-based authorization, a multi-type catalogue, transactional borrow/return handling, dashboards, analytics, an audit trail, and reporting —

built on a conventional, well-separated Spring Boot architecture, with every design decision traceable back to the alternatives that were explicitly considered and rejected (Section 6). All ten functional and seven non-functional requirements set at project start are delivered and evidenced (Section 4); all six defects identified across development and the pre-submission QA pass are closed and verified (Section 15); and the system is documented end to end — architecture, security, database, frontend, testing, and deployment — to the same standard it was built to.

The recommendations in Section 19 are not outstanding obligations but a deliberate roadmap: the places a production-hardened second iteration would extend a system that, as delivered, meets its stated objectives in full.

Appendix A: Complete API Endpoint Index

All 48 documented endpoints, alphabetically by resource group:

Method	Path	Authority
GET	/api/analytics/dashboard	Admin, Librarian
GET	/api/analytics/trends	Admin, Librarian
GET	/api/analytics/top-books	Admin, Librarian
GET	/api/analytics/top-readers	Admin, Librarian
GET	/api/analytics/overdue	Admin, Librarian
GET	/api/audit	Admin
GET	/api/audit/{id}	Admin
GET	/api/auth/csrf	Public
POST	/api/auth/login	Public
POST	/api/auth/logout	Authenticated
GET	/api/auth/me	Authenticated
GET	/api/books	Authenticated
GET	/api/books/{id}	Authenticated
POST	/api/books	Admin, Librarian
PUT	/api/books/{id}	Admin, Librarian
DELETE	/api/books/{id}	Admin, Librarian
GET	/api/borrow-records	Admin, Librarian
GET	/api/borrow-records/my	Authenticated
POST	/api/borrow-records	Admin, Librarian

Method	Path	Authority
POST	/api/borrow-records/{id}/return	Admin, Librarian
GET	/api/dashboard	Admin, Librarian
GET	/api/librarian/dashboard	Admin, Librarian
GET	/api/librarians	Admin
GET	/api/librarians/{id}	Admin
POST	/api/librarians	Admin
PUT	/api/librarians/{id}	Admin
DELETE	/api/librarians/{id}	Admin
GET	/api/magazines	Authenticated
GET	/api/magazines/{id}	Authenticated
POST	/api/magazines	Admin, Librarian
PUT	/api/magazines/{id}	Admin, Librarian
DELETE	/api/magazines/{id}	Admin, Librarian
GET	/api/newspapers	Authenticated
GET	/api/newspapers/{id}	Authenticated
POST	/api/newspapers	Admin, Librarian
PUT	/api/newspapers/{id}	Admin, Librarian
DELETE	/api/newspapers/{id}	Admin, Librarian
GET	/api/profile	Authenticated
GET	/api/reports/inventory	Admin, Librarian
GET	/api/reports/borrowing	Admin, Librarian

Method	Path	Authority
GET	/api/reports/overdue	Admin, Librarian
GET	/api/reports/students	Admin, Librarian
GET	/api/student/dashboard	Admin, Librarian, Student
GET	/api/students	Admin, Librarian
GET	/api/students/{id}	Admin, Librarian
POST	/api/students	Admin, Librarian
PUT	/api/students/{id}	Admin, Librarian
DELETE	/api/students/{id}	Admin, Librarian

Appendix B: Glossary

Term	Meaning
BCrypt	A one-way password-hashing algorithm; only the resulting hash is ever stored, never the plaintext password.
CSRF	Cross-Site Request Forgery — an attack where a malicious site submits state-changing requests using a victim's existing session; mitigated here via a cookie/header token exchange.
DTO	Data Transfer Object — a class shaped for API request/response payloads, deliberately separate from JPA entities to avoid leaking persistence internals.
JPA / Hibernate	Java Persistence API — the specification for object-relational mapping; Hibernate is the implementation Libris uses to map entities to MySQL tables.
JWT	JSON Web Token — a token-based auth mechanism considered and deferred in favour of server-side sessions (Section 6.1).
MPA	Multi-Page Application — an architecture where navigation triggers full page loads, as opposed to a client-side-routed single-page application.
ddl-auto	A Hibernate setting controlling automatic schema generation; update alters existing tables, create-drop recreates the schema each run (used in tests).
MockMvc	A Spring testing utility that simulates HTTP requests against controllers without starting a real server.
RBAC	Role-Based Access Control — the authorization model Libris uses, built on the Admin/Librarian/Student role enum.
XSRF-TOKEN	The cookie name used by CookieCsrfTokenRepository to expose the CSRF token to frontend JavaScript.